HACKEN

Blockchain Protocol Security Analysis Report

Customer: Push Chain

Date: 29/06/2026

We express our gratitude to the PushChain team for the collaborative engagement that enabled the execution of this Blockchain Protocol Security Assessment.

Push Chain is a shared-state Layer 1 that brings together Cosmos-style blockchain infrastructure with full Ethereum app compatibility, so developers can build in familiar tools while the chain coordinates activity across networks. Rather than connecting chains one bridge at a time, Push acts as a central hub where cross-chain actions are observed, agreed on, and executed—enabling universal applications that can reach users and assets on many chains from one place.

Document

Name	Blockchain Protocol Review and Security Analysis Report for PushChain
Audited By	Tanuj Soni, Alfredo Ortega
Approved By	Ece Örsel
Website	https://push.org/
Changelog	15/05/2026 - Preliminary Report
Changelog	05/06/2026 - Second Preliminary Report
Changelog	29/06/2026 - Final Report
Platform	PushChain
Language	Golang
Tags	Cosmos SDK, EVM compatible, Cross-chain hub
Methodology	https://docs.hacken.io/methodologies/blockchain-protocols

Review Scope

Repository	https://github.com/pushchain/push-chain-node
Initial Commit	`e6b7c77`
Final Commit	`0648551`
Repository	https://github.com/pushchain/push-chain-evm/
Initial Commit	`9570d87`
Final Commit	`96231e7`

Audit Summary

The system users should acknowledge all the risks summed up in the risks section of the report

55	44	10	1
Total Findings	Resolved	Accepted	Mitigated

Findings by Severity

Severity	Count
Critical	0
High	2
Medium	8
Low	32

Vulnerability	Severity	Status
F-2026-16597 - Arbitrary memory allocation at UniversalClient	High	Fixed
F-2026-16963 - [PUSHCHAIN REPORTED] Issue 4 — EVM TX resolver marks “Tx not found” as reverted (double-spend edge case)	High	Fixed
F-2026-16738 - ExecuteInboundFundsAndPayload State Desync	Medium	Fixed
F-2026-16740 - Solana RPC Signature Order Misassumption	Medium	Fixed
F-2026-16960 - [PUSHCHAIN-REPORTED] Issue 1: RPC failover retries same endpoint under concurrent load	Medium	Fixed
F-2026-16961 - [PUSHCHAIN REPORTED] Issue 2 — Fund migration failure due to hardcoded GasLimit and missing L1GasFeeBuffer	Medium	Fixed
F-2026-16962 - [PUSHCHAIN-REPORTED] Issue 3 — Fund migration broadcast retry storm after peer already migrated funds	Medium	Fixed
F-2026-16966 - [PUSHCHAIN-REPORTED] Issue 7 — Expiry sweeper votes REVERT based on push chain state, not observed chain state and can result into false voting	Medium	Fixed
F-2026-17087 - [PUSHCHAIN-REPORTED] Issue 9 - Inbound-revert outbound ID collides when a single source tx contains multiple Inbounds	Medium	Fixed
F-2026-16998 - Hardcoded EOA as owner of every system-contract ProxyAdmin	Medium	Accepted
F-2026-16039 - Inbound ballot identity derived from full serialized Inbound	Low	Fixed
F-2026-16599 - DoS via Slowloris attack on API server at UniversalClient	Low	Fixed
F-2026-16632 - Outbound ballot identity is coupled to full OutboundObservation encoding	Low	Fixed
F-2026-16642 - Pending inbound index can outlive terminal ballot state	Low	Fixed

Vulnerability	Severity	Status
F-2026-16874 - Coordinator retains prior validator set when refresh RPC fails	Low	Fixed
F-2026-16875 - Event confirmers advance confirmation depth without execution success	Low	Fixed
F-2026-16939 - Unbounded frame allocation in TSS libp2p readFramed	Low	Fixed
F-2026-16944 - Event Skipping via Pagination Failure in Solana Event Listener	Low	Fixed
F-2026-16964 - [PUSHCHAIN REPORTED] Issue 5 — SVM TX resolver lacks a way to differentiate invalid signature vs transient simulation failure	Low	Fixed
F-2026-16965 - [PUSHCHAIN REPORTED] Issue 6 — Reverted txs can get stuck due to architecture (failure visibility limited to signer set)	Low	Fixed
F-2026-16967 - [PUSHCHAIN-REPORTED] Issue 8 - MsgExecutePayload rejects when precomputed UEA is pre-funded but not deployed	Low	Fixed
F-2026-16991 - Stale active universal validators inflate ballot quorum and can deadlock finalization	Low	Fixed
F-2026-16992 - Removing an active universal validator bypasses the ongoing-TSS guard used elsewhere	Low	Fixed
F-2026-16993 - Votes are accepted on pending ballots without checking block height expiry	Low	Fixed
F-2026-17022 - Case-sensitive token keys versus case-insensitive PRC20 lookup and EIP-55 observer addresses	Low	Fixed
F-2026-17025 - isContractDeployed treats any account with a non-empty code hash as a deployed contract	Low	Fixed
F-2026-17038 - Genesis export/import omits pending TSS event index	Low	Fixed
F-2026-17041 - Fund migration ballot key uses raw txHash string	Low	Fixed
F-2026-17043 - usigverifier Ed25519 precompile verifies signature over ASCII hex, not the raw message	Low	Fixed
F-2026-17736 - Derived call can commit state even when commit=false	Low	Fixed
F-2026-17738 - Reverted derived execution still emits events and mutates block bloom	Low	Fixed
F-2026-17740 - KV indexer path misses derived transactions	Low	Fixed
F-2026-17745 - Derived transactions share a constant txIndex value	Low	Fixed
F-2026-17752 - Derived RPC transaction uses GasUsed as gas field	Low	Fixed
F-2026-17754 - TraceTransaction mixes Cosmos and Ethereum index domains	Low	Fixed
F-2026-17775 - Inconsistent log reporting for failed derived transactions across JSON-RPC endpoints	Low	Fixed
F-2026-16658 - Threshold signing ballots require all process participants	Low	Accepted
F-2026-16659 - Event-driven outbound observations set success unconditionally	Low	Accepted
F-2026-16868 - Fixed gas limit and fee for all vote transactions	Low	Accepted

Vulnerability	Severity	Status
F-2026-16876 - Observers bind to registry-supplied gateway addresses without on-chain contract integrity checks	Low	Accepted
F-2026-16940 - Non-idempotent ordering between failure vote submission and local persistence in the expiry sweeper	Low	Accepted
F-2026-17758 - Missing upstream security patch train after fork point	Low	Mitigated
F-2026-16598 - Unsafe Status Modification Order in MarkBallotExpired() and MarkBallotFinalized()	Info	Fixed
F-2026-16616 - Chain Metadata Oracle Using First Vote Instead of Median Aggregation	Info	Fixed
F-2026-16648 - Default registry administrator in DefaultParams	Info	Fixed
F-2026-16867 - Logging and standard output disclose sensitive material	Info	Fixed
F-2026-16877 - Chains.Start returns successfully when the Push native chain client is not attached	Info	Fixed
F-2026-16994 - Nil Network on MsgUpdateUniversalValidator panics in validation and in the handler	Info	Fixed
F-2026-16996 - Unwired v2 migration overwrites every universal validator network identity with placeholders	Info	Fixed
F-2026-17024 - Nil nested configs on add/update messages cause panics in validation and logging	Info	Fixed
F-2026-17035 - Unpaginated query walks and full-collection scan in GetTokenConfigByPRC20	Info	Fixed
F-2026-16614 - Centralized registry and Universal Validator roster	Info	Accepted
F-2026-16619 - ExecutePayload and binding of UniversalAccountId.owner to the signer	Info	Accepted
F-2026-16655 - Fund migration votes pass txHash without normalization	Info	Accepted
F-2026-17040 - TSS event status updates scan entire event store	Info	Accepted

Documentation quality

- The root `readme.md` provides a concise product overview, environment prerequisites, local network startup guidance, and a repository layout. It does not articulate functional requirements, explicit security assumptions, or normative guarantees for cross-chain execution paths.
- Documentation for the custom modules (`x/uexecutor`, `x/uregistry`, `x/uvalidator`, `x/utss`) is largely introductory. It does not systematically document message lifecycles, failure and recovery behavior, voting or quorum semantics, or administrative control boundaries to a level typically expected for independent protocol review.
- A consolidated technical specification covering trust model, finality and confirmation policy, upgrade authority, and state-transition invariants—is not present as a standalone deliverable. Protocol behavior is inferred primarily from source code and protobuf definitions the latter describe data shapes but do not substitute for a full protocol specification. `chain_metadata.json` includes placeholder or non-specific external references, which is consistent with metadata that has not been finalized for production publication.

Code quality

- The implementation builds on established Cosmos SDK and Cosmos EVM components in a conventional manner. Push-specific EVM system contracts are integrated using widely used proxy patterns, with contract bytecode embedded in the registry module rather than distributed as a separately versioned contract package.
- Some module scaffolding remains (for example unwired or placeholder migrations and example chain metadata). If activated or merged without additional review, such artifacts can increase operational and maintenance risk.
- The universal modules and Universal Client introduce substantial custom logic. Cross-cutting concerns including hash normalization, ballot or observation identity, and oracle-related updates —are not uniformly enforced across all components, which can complicate reasoning about end-to-end behavior.
- The repository provides a workable development setup: `Makefile`, Docker-oriented tooling, local and testnet-oriented scripts, and stated prerequisites (Go 1.23+, Docker, `jq`).

Architecture quality

- **Scalability:** The network is implemented as a single CometBFT-based L1 without sharding. Capacity is bounded by block limits, EVM execution cost, and off-chain operator throughput. Certain on-chain bookkeeping paths rely on linear scans over growing state, which may become a scaling consideration as history accumulates.
- **Decentralization:** Consensus follows a standard validator model. Cross-chain operation additionally depends on a distinct universal-validator role, registry-defined routing and asset policy, and EVM-layer upgrade control for system contracts. Together, these introduce administrative and operational trust assumptions beyond the base staking layer.
- **Interoperability:** The design targets interoperability via IBC, EVM interfaces, CosmWasm, and hub-style connectivity to external EVM and Solana networks through Universal Client. Cross-chain settlement is attestation-based and configuration-dependent rather than light-client–verified on a per-event basis.
- **Resilience:** End-to-end resilience of cross-chain flows is not fully evidenced by the reviewed materials. Operational availability depends on Universal Client processes, external RPC reliability,

and alignment between staking-derived validator state and universal-validator registry state. EVM system-contract upgrade authority represents a concentrated dependency in the overall architecture.

Table of Contents

System Overview

Push Chain is a Cosmos SDK Layer 1 on CometBFT with native EVM and CosmWasm support. The node binary is `pchaind` the native token is `upc`. Alongside standard Cosmos, EVM, IBC, and token-factory modules, the chain adds four custom modules for universal cross-chain apps: registry, validator voting, on-chain execution, and threshold-signing coordination.

External chains connect through a hub model: operators run `puniversald` to observe events, validators vote them into consensus, Push executes the workflow (including EVM), and signed transactions complete actions on destination chains. Selected user and validator transactions can be processed without gas fees when they use only approved message types.

Custom Modules

- **Universal Client (`universalClient`)**
  Off-chain service run by universal validators alongside `pchaind`. Watches external chains (EVM and Solana), submits vote transactions to Push, and collectively signs outbound transactions via threshold cryptography so no single operator holds full signing authority. Persists the chain-specific state locally and coordinates TSS sessions between validators.
- **Universal Registry (`x/uregistry`)**
  Defines which external chains, tokens, and gateways Push recognizes. Administrators manage this configuration; execution modules reject operations on unsupported or disabled routes.
- **Universal Validator (`x/uvalidator`)**
  Maintains the universal validator set and runs ballot voting on cross-chain observations. When enough validators agree, results are passed to the executor and TSS modules.
- **Universal Executor (`x/uexecutor`)**
  Runs universal workflows on Push: processing inbounds from other chains, tracking transaction status, executing user payloads, and handling outbounds. Integrates with the EVM layer for contracts and account logic.
- **Universal TSS (`x/utss`)**
  Coordinates threshold signing and fund migration when validator keys rotate. On-chain votes align with off-chain signing performed by Universal Client operators.

EVM Precompiles

Push extends the standard EVM precompile set so Solidity can call into Cosmos: bech32, p256, staking, distribution, IBC transfer, bank, gov, slashing, and evidence.

Push also adds `usigverifier`, which lets contracts verify signatures from supported external chains (e.g. Ed25519).

Risks

- Core cross-chain contracts can be upgraded outside normal Cosmos governance; misuse could change system behavior and affect bridged assets.
- Settlement still relies on validator agreement, with limited automatic protection against incorrect votes.
- Key-management steps that need full participant agreement can stall if someone does not take part.
- Events may be recorded before they are fully settled elsewhere, which can lead to later mismatches with source chains.
- Accounts are tied to each source chain and owner, not automatically to one identity across all chains.

Findings

Vulnerability Details

F-2026-16597 - Arbitrary memory allocation at UniversalClient - High

Description:

The `decodePayload` function in the Solana (SVM) transaction builder at `universalclient/chains/svm/tx_builder.go` is vulnerable to a memory exhaustion-based denial-of-service (DoS) attack. The function reads `accountsCount` (`uint32`) and `ixDataLen` (`uint32`) directly from an untrusted payload byte slice and immediately uses these values to allocate slices via `make()` without any upper-bound validation. A maliciously crafted payload with extremely large values can force the Go runtime to attempt allocating gigabytes of memory, triggering an "out of memory" panic that crashes the validator process.

In this code:

```
func decodePayload(payload []byte) ([]GatewayAccountMeta, []byte, u
int8, [32]byte, error) {
var targetProgram [32]byte
// Minimum payload: accounts_count(4) + ix_data_len(4) + instructio
n_id(1) + target_program(32) = 41
if len(payload) < 41 {
return nil, nil, 0, targetProgram, fmt.Errorf("payload too short: %
d bytes (minimum 41)", len(payload))
}
offset := 0
accountsCount := binary.BigEndian.Uint32(payload[offset : offset+4]
)
offset += 4

accounts := make([]GatewayAccountMeta, accountsCount)
```

We can see how the amount of accounts (`accountsCount`) is never validated for either maximum or minimum values.
This could cause an arbitrary memory allocation that will ultimately cause a Out-of-Memory crash in the client.

Also, no payload validation is done before parsing of the fields.

Status: Fixed

Classification

Impact Rate: 4/5

Likelihood Rate: 4/5

Severity: High

Recommendations

Resolution: This issue has been addressed and resolved in commit 17581bf.

F-2026-16963 - [PUSHCHAIN REPORTED] Issue 4 — EVM TX resolver marks “Tx not found” as reverted (double-spend edge case) - High

Description: If a tx is not found on chain after broadcast, it is marked as reverted. This is unsafe if the signed tx was merely dropped from mempool due to gas spikes; a malicious UV could rebroadcast later when gas is lower (if nonce not yet consumed).

Status: Fixed

Classification

Impact Rate: 5/5

Likelihood Rate: 2/5

Severity: High

Recommendations

Remediation: Retry broadcast and only revert once nonce is consumed

Resolution: This issue has been addressed and resolved in commit 78a44a4.

F-2026-16738 - ExecuteInboundFundsAndPayload State Desync - Medium

Description:

The vulnerability resides in the `ExecuteInboundFundsAndPayload` function within the `x/uexecutor` module. The execution flow performs an external smart contract call (`CallExecuteUniversalTx`) which modifies the EVM state, followed by a local state update (`DeductGasFeesFromReceipt`). If the gas fee deduction fails, the function catches the error, marks the Universal Transaction (UTX) status as `FAILED`, and returns `nil` to the Cosmos SDK runtime.

As function returns `nil`, Cosmos SDK doesn't revert it. There is no rollback, compensation, or state reconciliation mechanism. This creates a critical divergence between the Cosmos SDK module state and the EVM state, leading to unrecoverable accounting discrepancies and potential fund misallocation.

This behavior violates the principle of atomicity in transaction processing. By returning `nil` instead of propagating the error, the Cosmos SDK commits the transaction, thereby finalizing the state changes made by `CallExecuteUniversalTx` on the EVM side. However, the Cosmos module state records the transaction as failed. This creates a critical divergence:

1. **Irreversible State Change:** The EVM execution (e.g., token transfers, contract interactions) is permanently committed on the target chain.
2. **Accounting Mismatch:** The source chain believes the transaction failed, potentially leaving funds stranded or creating double-spend vulnerabilities if the user retries the transaction.

This flaw can be exploited by any user submitting a Universal Transaction where the EVM execution succeeds but the subsequent gas deduction fails (e.g., due to insufficient balance in the fee account or parsing errors). The result is a loss of funds or system integrity, as the executed payload has no corresponding valid receipt or refund path.

Status: Fixed

Classification

Impact Rate: 4/5

Likelihood Rate: 1/5

Severity: Medium

Recommendations

Remediation: To mitigate this vulnerability, implement one of the following strategies to ensure atomicity and state consistency:

Propagate Errors for Automatic Revert: Modify `ExecuteInboundFundsAndPayload` to return the error from `DeductGasFeesFromReceipt` directly to the Cosmos SDK runtime. This will cause the entire transaction, including the EVM state changes, to be reverted, ensuring that no state is committed unless all steps succeed.

Implement Pre-Validation: Move the gas fee deduction logic or balance checks before the `CallExecuteUniversalTx` execution. Ensure that sufficient funds and valid conditions are verified prior to initiating the irreversible EVM call.

Resolution: This issue has been addressed and resolved in commit b890c7f.

F-2026-16740 - Solana RPC Signature Order Misassumption - Medium

Description:

The `processSlotRange` function incorrectly terminates prematurely and skip critical inbound events, potentially stalling cross-chain operations.

2. Detailed Explanation

The vulnerability is located in the `processSlotRange` function within `universalClient/chains/svm/event_listener.go`. The function is responsible for scanning a specific range of slots (`fromSlot` to `toSlot`) to identify and process transactions associated with a gateway address.

Technical Analysis

The implementation iterates through the results of the `GetSignaturesForAddress` RPC call using the following logic:

```
for _, sig := range signatures {
if sig.Slot < fromSlot {
continue
}
if sig.Slot > toSlot {
break
}
// ... process transaction ...
}
```

The flaw lies in the use of the `break` statement when `sig.Slot > toSlot`. According to the official Solana RPC specification, the `getSignaturesForAddress` method returns signatures in **reverse chronological order** (newest first/descending slot order), but even this ordering is not guaranteed in all implementations (https://github.com/solana-labs/solana/issues/22456)

Because the list is sorted from highest slot to lowest slot, the first element encountered is the most recent. If the RPC returns any signature with a slot number greater than `toSlot` (which is obtained using `GetLatestSlot`() but could be outdated if new signatures arrive between calls), the `break` condition is triggered immediately. This terminates the loop and ignores all subsequent signatures in the slice, even those that fall perfectly within the `[fromSlot, toSlot]` range.

Impact

This bug creates a critical failure in the event detection pipeline. Since the system updates the `lastProcessedSlot` in the database after calling `processSlotRange`, any signatures skipped due to the premature `break` are effectively marked as processed and will never be revisited.

The consequences include:

- **Stalled Cross-Chain Operations:** Inbound events from the Solana chain to the Push Chain will be missed.
- **Loss of State Synchronicity:** The validator's internal state will diverge from the actual on-chain state.
- **Operational Failure:** User requests to move assets or trigger actions will remain undetected, leading to a complete breakdown of the bridge/gateway functionality.

Status: Fixed

Classification

Impact Rate: 4/5

Likelihood Rate: 4/5

Severity: Medium

Recommendations

Remediation: Pre-order the signatures so the algorithm do not depends on the Solana ordering.

Resolution: This issue has been addressed and resolved in commit 35bd9b6.

F-2026-16960 - [PUSHCHAIN-REPORTED] Issue 1: RPC failover retries same endpoint under concurrent load - Medium

Description:

Both EVM and SVM clients implement endpoint failover via `executeWithFailover` using a shared atomic counter `rc.index`. Because the counter is incremented *inside* the retry loop, concurrent callers can advance the same counter between retries. This can cause a single caller to retry the same failing endpoint and never try the alternate endpoint.

Why it matters

Under load, failover can silently degrade into "repeat the bad endpoint," increasing the probability of request failure and cascading component failures (listener/confirmers/oracle/signer, etc.).

Evidence / permalinks (audit-main)

- EVM: https://github.com/pushchain/push-chain-node/blob/audit-main/universalClient/chains/evm/rpc_client.go#L107
- SVM: https://github.com/pushchain/push-chain-node/blob/audit-main/universalClient/chains/svm/rpc_client.go#L122

Status: Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 3/5

Severity: Medium

Recommendations

Remediation: Snapshot the counter once before the retry loop, then index by `startIndex + attempt` so each caller deterministically rotates through all endpoints.

```
startIndex := atomic.AddUint64(&rc.index, 1) - 1
for attempt := 0; attempt < maxAttempts; attempt++ {
client := clients[(startIndex + uint64(attempt)) % uint64(len(clien
ts))]
// ... try client, break on success ...
}
```

Resolution: This issue has been addressed and resolved in commit 4706e90.

F-2026-16961 - [PUSHCHAIN REPORTED] Issue 2 — Fund migration failure due to hardcoded GasLimit and missing L1GasFeeBuffer - Medium

Description:

`InitiateFundMigration` records gas parameters used by UV to broadcast the migration transfer.

- **Gas price** is dynamic (from chain meta oracle).
- **Gas limit** is hardcoded to `21000` via `nativeTransferGasLimit`.
- There is **no L1 data fee buffer** concept for OP-Stack chains (Base/Optimism), where fees include L2 execution + L1 calldata posting costs.

Why it matters

`21000` is not universal across EVM networks (e.g., some L2s require higher gas for native transfer).

OP-Stack requires budgeting for L1 data fee; without it, tx may be rejected or the relayer subsidizes unexpectedly.

Evidence / permalinks (audit-main)

- Constant: https://github.com/pushchain/push-chain-node/blob/audit-main/x/utss/keeper/msg_initiate_fund_migration.go#L11
- Use site: https://github.com/pushchain/push-chain-node/blob/audit-main/x/utss/keeper/msg_initiate_fund_migration.go#L88

Status: Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 2/5

Severity: Medium

Recommendations

Remediation: Move `gas_limit` and `l1_gas_fee_buffer` into an admin-modifiable param in `UniversalCore` contract on push chain, and include both in the migration record/event.

- **Core:** fetch gas params from universalCore for that chain in `MsgInitiateFundMigration` and emit those out in the `FundMigration` event that UV can use.
- **UV:** honor `L1GasFeeBuffer` when computing total fee allowance for OP-Stack.

Resolution: This issue has been addressed and resolved in commit cc7e027.

F-2026-16962 - [PUSHCHAIN-REPORTED] Issue 3 — Fund migration broadcast retry storm after peer already migrated funds - Medium

Description:

All UV nodes race to broadcast the signed fund migration tx. The first to land drains the old TSS address to zero. Others then attempt to assemble a tx by fetching current balance and fail with "insufficient balance for gas," then retry every tick.

Example log:

```
WRN failed to assemble fund migration tx, will retry next tick
error="insufficient balance for gas: balance=0 gasCost=316875657000
l1GasFee=0"
chain=eip155:11155111
component=txbroadcaster
event_id=e57ab88f58e2f154c9f6fd6210b23c4d0db26bec2a9593a9ce134dbb70
b316bd
```

Evidence / permalinks (audit-main)

- Retry log site: https://github.com/pushchain/push-chain-node/blob/audit-main/universalClient/tss/txbroadcaster/evm.go#L121
- Function: https://github.com/pushchain/push-chain-node/blob/audit-main/universalClient/tss/txbroadcaster/evm.go#L73

Status: Fixed

Classification

Impact Rate: 4/5

Likelihood Rate: 2/5

Severity: Medium

Recommendations

Remediation: Use the **balance/value assumptions from signing time** (or signed payload) rather than re-fetching balance at assembly time, and/or add a "already migrated" detection path to stop retries.

Resolution: This issue has been addressed and resolved in commit 58ed01b.

F-2026-16966 - [PUSHCHAIN-REPORTED] Issue 7 — Expiry sweeper votes REVERT based on push chain state, not observed chain state and can result into false voting - Medium

Description:

Each UV node runs an expirysweeper that:

- Finds CONFIRMED events whose expiry block has passed.
- Votes failure (`voteOutboundFailure` / `voteFundMigrationFailure`) on Push Chain.
- Marks events REVERTED locally.

Why it matters

- Reset/lagging nodes keep stale local state until timer fires; then they cast redundant votes.
- Non-signing committee nodes can vote failure even if the tx succeeded on-chain but inbound observation is temporarily broken → split votes / false failures.

Evidence / permalinks (audit-main)

- Sweeper entry: https://github.com/pushchain/push-chain-node/blob/audit-main/universalClient/tss/expirysweeper/sweeper.go#L80
- Outbound revert vote: https://github.com/pushchain/push-chain-node/blob/audit-main/universalClient/tss/expirysweeper/sweeper.go#L125
- Fund migration revert vote: https://github.com/pushchain/push-chain-node/blob/audit-main/universalClient/tss/expirysweeper/sweeper.go#L161

Status: Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 2/5

Severity: Medium

Recommendations

Remediation: Remove expirysweeper from the voting path and make settlement observation-based for outbound and fund migration events.

Resolution: This issue has been addressed and resolved in commit 65b529a.

F-2026-16998 - Hardcoded EOA as owner of every system-contract ProxyAdmin - Medium

Description:

Push EVM system contracts deployed from the universal registry sit behind transparent proxies whose `ProxyAdmin` contracts follow an OpenZeppelin-style `Ownable` pattern. At genesis, the owner of each `ProxyAdmin` is set from a single hardcoded externally owned account constant. That account can invoke `upgradeAndCall` on every such proxy without going through Cosmos governance. Module messages such as `MsgUpdateParams` only govern registry metadata (parameters, chain and token configuration) they do not constrain who may upgrade the EVM execution surface. The trust model therefore splits chain configuration authority from upgrade authority for cross-chain messaging contracts, with the latter resting on one EOA and normal EVM transactions.

File: `push-chain-node/x/uregistry/types/constants.go` ·

```
const PROXY_ADMIN_OWNER_ADDRESS_HEX = "0xa96CaA79eb2312DbEb0B8E93c1
Ce84C98b67bF11"
```

File: `push-chain-node/x/uregistry/keeper/genesis.go` ·

```
func deployProxyAdminContract(ctx context.Context, evmKeeper types.
EVMKeeper, proxyAdminContractAddress string, runtimeBytecode []byte
) {
sdkCtx := sdk.UnwrapSDKContext(ctx)
proxyAdminAddress := common.HexToAddress(proxyAdminContractAddress)
owner := common.HexToAddress(types.PROXY_ADMIN_OWNER_ADDRESS_HEX)
// ...
evmKeeper.SetState(sdkCtx, proxyAdminAddress, common.Hash{}, common
.LeftPadBytes(owner.Bytes(), 32))
}
```

Storage slot zero is written with the owner address (standard `Ownable.owner` layout for the embedded proxy-admin bytecode), granting that address upgrade rights for the associated proxies.

File: `push-chain-node/x/uregistry/types/constants.go` ·

```
var SYSTEM_CONTRACTS = map[string]ContractAddresses{
"UNIVERSAL_CORE": {
Address: "0x00000000000000000000000000000000000000C0",
ProxyAdmin: "0xf2000000000000000000000000000000000000c0",
Implementation: "0xF1000000000000000000000000000000000000c0",
},
// ... UNIVERSAL_BATCH_CALL, UNIVERSAL_GATEWAY_PC, RESERVED_0..2
}
```

File: `push-chain-node/x/uregistry/keeper/genesis.go` ·

```
func deploySystemContracts(ctx context.Context, evmKeeper types.EVM
Keeper, systemContracts map[string]types.ContractAddresses) {
// ...
```

```
for _, name := range names {
contract := systemContracts[name]
// ...
deployProxyAdminContract(ctx, evmKeeper, contract.ProxyAdmin, bytec
odes.ADMIN_RUNTIME)
deployImplementationContract(ctx, evmKeeper, contract.Implementatio
n, bytecodes.IMPL_RUNTIME)
deployProxyContract(ctx, evmKeeper, contract.Address, contract.Prox
yAdmin, contract.Implementation, bytecodes.PROXY_RUNTIME)
}
}
```

The same constant is referenced from `x/uexecutor` for related system-contract deployment, so upgrade authority is unified across the universal execution surface at the EVM layer.

Compromise, loss, or misuse of the owner key allows arbitrary replacement of implementations for core and reserved system contracts. An attacker could redirect vault balances, alter gateway or batch-call behavior, or pre-empt reserved proxy slots. Cosmos governance cannot revoke or rotate this authority on its own; recovery depends on the EOA performing `transferOwnership` or equivalent EVM actions. This centralization exceeds the blast radius of Cosmos-level registry admin parameters documented elsewhere, because it affects live contract code rather than configuration records alone.

Status: Accepted

Classification

Impact Rate: 3/5

Likelihood Rate: 2/5

Severity: Medium

Recommendations

Remediation:

- Initialize `ProxyAdmin` ownership with a multisig or other contract-governed account rather than a single EOA before production launch.
- Tie upgrades to an on-chain path that reflects governance decisions (for example an adapter contract callable only after a passed proposal is verified).
- Add a timelock on upgrade actions so operators and the community can react to unexpected changes.
- Publish the production ownership model, rotation procedure, and post-genesis transfer plan as launch criteria.
- Monitor that every deployed `ProxyAdmin` owner matches the intended governance-controlled set after each upgrade window.

F-2026-17087 - [PUSHCHAIN-REPORTED] Issue 9 - Inbound-revert outbound ID collides when a single source tx contains multiple Inbounds - Medium

Description:

When the chain decides an inbound cannot be executed (validation failure post-quorum), it constructs a synthetic `INBOUND_REVERT` outbound to refund the user on the source chain. That outbound ID is generated by `GetOutboundRevertId`, and the same ID is later used as the subTxId on the source-chain gateway / vault calls that perform the revert.

The current implementation derives the ID from only (sourceChain, txHash, "REVERT"):

```
// x/uexecutor/types/keys.go
func GetOutboundRevertId(sourceChain string, inboundTxHash string)
string {
data := fmt.Sprintf("%s:%s:REVERT", sourceChain, inboundTxHash)
hash := sha256.Sum256([]byte(data))
return hex.EncodeToString(hash[:])
}
```

If a single source-chain transaction emits multiple bridge events (different log_index values) and more than one of them needs to be reverted, every revert outbound for that tx maps to the same ID. The two inbound observations are correctly separated by log_index at every other point in the system (UTX key, ballot, pending-inbound set), but the revert ID is not.

Status: Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 2/5

Severity: Medium

Recommendations

Remediation:

Add `log_index` to the digest input so the revert ID is unique per inbound event. Concretely:

```
// x/uexecutor/types/keys.go
func GetOutboundRevertId(sourceChain, inboundTxHash, logIndex strin
g) string {
data := fmt.Sprintf("%s:%s:%s:REVERT", sourceChain, inboundTxHash,
logIndex)
hash := sha256.Sum256([]byte(data))
```

```
return hex.EncodeToString(hash[:])
}
```

And update the single caller to pass `inbound.LogIndex`:

// x/uexecutor/keeper/build_revert_outbound.go

```
Id: types.GetOutboundRevertId(inbound.SourceChain, inbound.TxHash,
inbound.LogIndex)
```

This brings the revert key in line with the inbound UTX key (GetInboundUniversalTxKey already uses `source_chain:tx_hash:log_index`) and the rescue key (GetRescueFundsOutboundId already uses pcCaip:txHash:logIndex:RESCUE).

Resolution: This issue has been addressed and resolved in commit e46574f.

F-2026-16039 - Inbound ballot identity derived from full serialized

Inbound - Low

Description:

Inbound voting derives its ballot identifier from the complete protobuf encoding of the `Inbound` message, whereas the on-chain Universal Transaction identifier uses only a deterministic digest of `source_chain`, `tx_hash`, and `log_index`. Universal Validators must therefore marshal identical `Inbound` payloads for the same logical bridge event. Divergence in optional fields, decoding, amounts, or metadata yields distinct ballot identifiers, fragments votes across ballots, and can prevent quorum from forming even when operators agree off-chain.

The implementation exposes the ballot key as `hex.EncodeToString` of `Inbound.Marshal()`, not as a separate hash of those bytes; the liveness concern is unchanged: voting identity is coupled to the full serialized message

The Universal Transaction key hashes a minimal string tuple; the inbound ballot key uses the marshaled message body.

```
func GetInboundUniversalTxKey(inbound Inbound) string {
data := fmt.Sprintf("%s:%s:%s", inbound.SourceChain, inbound.TxHash
, inbound.LogIndex)
hash := sha256.Sum256([]byte(data))
return hex.EncodeToString(hash[:]) // hash[:] converts [32]byte → [
]byte
}

func GetInboundBallotKey(inbound Inbound) (string, error) {
bz, err := inbound.Marshal()
if err != nil {
return "", err
}
return hex.EncodeToString(bz), nil
}
```

The inbound ballot path calls `GetInboundBallotKey` before `VoteOnBallot`. The keeper records that the ballot uses the inbound payload as observed, so differing field data produces distinct votes by construction.

```
ballotKey, err := types.GetInboundBallotKey(inbound)
if err != nil {
return false, false, err
}
```

End users may see funds appear stalled at the gateway while explorers still show activity, because votes never consolidate onto one ballot identifier. Operators incur longer incidents spent diffing raw payloads and client versions instead of isolating business-logic faults. At the protocol level, bridge finalization depends on strict software homogeneity across the validator set, not merely on an honest majority agreeing on the same chain event.

Status: Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 1/5

Severity: Low

Recommendations

Remediation: Define a minimal, versioned canonical observation identifier used only for voting, and derive the ballot key from that digest while retaining the full `Inbound` for execution after quorum. Publish a single reference encoder with conformance tests so every Universal Validator release emits identical vote keys for the same on-chain event. Document explicitly for integrators that identical transaction hashes do not imply identical inbound ballots under the current scheme.

Resolution: This issue has been addressed and resolved in commit 93de525.

F-2026-16599 - DoS via Slowloris attack on API server at UniversalClient - Low

Description:

The function `NewServer()` at `universalClient/api/server.go` initializes an http.Server without specifying any timeout configurations (such as `ReadTimeout`, `WriteTimeout`, `IdleTimeout`, or `ReadHeaderTimeout`). By using the default `http.Server` settings, the server is vulnerable to Slowloris attacks and other Denial of Service (DoS) vectors where an attacker can keep connections open indefinitely by sending data very slowly or not sending the request header at all, eventually exhausting the server's connection pool and making it unresponsive to legitimate traffic.(R)

The `http.Server` struct is created with only two fields populated:

No timeout fields are set, making all timeout values default to zero.

```
func (s *Server) Start() error {
// ...
ln, err := net.Listen("tcp", s.server.Addr)
// ...
go func() {
err := s.server.Serve(ln)
// ...
}()
return nil
}
```

The server binds to `0.0.0.0:<port>` (all interfaces) and serves indefinitely without timeout protections.

The Slowloris attack is defined as:
- Attacker sends: `GET /health HTTP/1.1\r\nHost: target:8080\r\nX-slow: a\r\n`
- Attacker sends one byte every 30 seconds to keep the connection open
- Attacker send as many connections as configured by the operative system, blocking the server from opening any valid connections.

Status: Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 2/5

Severity: Low

Recommendations

Remediation: On Linux, the default max amount of connection is a very high number (>1000000) thus making this attack slow and unlikely from outside a local network, but still possible.

Resolution: This issue has been addressed and resolved in commit 6f961a6.

F-2026-16632 - Outbound ballot identity is coupled to full OutboundObservation encoding - Low

Description:

The outbound voting key is derived from `utxId`, `outboundIndex`, and the full serialized `OutboundObservation`. This design binds voting identity to complete observation encoding rather than to a narrowly defined canonical representation of the underlying outbound fact.

As a result, validators can reach the same business conclusion while still deriving different ballot keys if their observation construction differs in non-essential fields. In that condition, votes fragment across multiple ballot identities and quorum formation may be delayed or prevented.

This failure mode is materially aligned with F-2026-16039 on inbound observation identity, with the same core risk expressed in the outbound voting path.

The key derivation routine serializes the entire observation payload and includes it in the hash preimage used for ballot identity.

```
func GetOutboundBallotKey(
utxId string,
outboundIndex string,
observedTx OutboundObservation,
) (string, error) {

bz, err := observedTx.Marshal()
if err != nil {
return "", err
}

data := append([]byte(utxId+":"+outboundIndex+":"), bz...)
hash := sha256.Sum256(data)

return hex.EncodeToString(hash[:]), nil
}
```

Because ballot identity depends on full observation materialization, any implementation variance in observation composition can alter the resulting key even when validators are observing the same outbound event.

Status: Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 1/5

Severity: Low

Recommendations

Remediation: Adopt a canonical voting digest that includes only consensus-critical outbound attributes and use this digest exclusively for ballot key derivation. Persist the full `OutboundObservation` only as supporting evidence after quorum is finalized.
To reduce recurrence risk:

- Publish a single normative specification for outbound observation construction.
- Maintain conformance vectors and deterministic test fixtures across validator implementations.

Resolution: This issue has been addressed and resolved in commit 93de525.

F-2026-16642 - Pending inbound index can outlive terminal ballot state - Low

Description:

`VoteInbound` adds an inbound to `PendingInbounds` before recording a ballot vote.Cleanup of `PendingInbounds` is tied to successful post-finalization paths that create a `UniversalTx`.If a ballot reaches a terminal non-pending status (for example, `EXPIRED`) without traversing those cleanup paths, the pending entry can remain indefinitely. At that point, further voting is rejected because ballot voting only accepts `PENDING` ballots.
Result: the ballot is terminal, but the inbound may still appear pending in executor state.

The inbound is inserted into `PendingInbounds` before ballot resolution and that cached state is then committed. Cleanup is performed later on post-finalization execution paths, and `RemovePendingInbound` is called from this file only. If a terminal state is reached outside that cleanup path, the pending state can remain while the ballot is already terminal.

The lock condition is explicit in the ballot keeper:
`File: x/uexecutor/keeper/msg_vote_inbound.go`

```
if ballot.Status != types.BallotStatus_BALLOT_STATUS_PENDING {
k.Logger().Warn("ballot is not in pending state, cannot vote",
"ballot_id", id,
"ballot_status", ballot.Status.String(),
"voter", voter,
)
return ballot, false, false, fmt.Errorf("ballot %s is already %s",
id, ballot.Status.String())
}
```

The keying model also differs by design: pending index entries use `GetInboundUniversalTxKey(inbound)` (hash of `source_chain:tx_hash:log_index`), while ballots use `GetInboundBallotKey(inbound)` (marshaled full inbound payload), as defined in `x/uexecutor/types/keys.go` This is valid behavior, but it makes recovery more complex once those two state views diverge.

Status: Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 1/5

Severity: Low

Recommendations

Remediation: Add explicit reconciliation hooks so that any inbound ballot transition to a terminal state that does not create a `UniversalTx` clear `PendingInbounds` entry. Define and document canonical inbound lifecycle transitions (`PENDING` → `PASSED` / `REJECTED` / `EXPIRED`) with required secondary-index side effects, ship expiry tuning, and reconciliation in the same release train, and add integration tests that assert terminal-ballot and pending-index consistency invariants. Until automated reconciliation is in place, document an interim operational recovery runbook.

Resolution: This issue has been addressed and resolved in commit cbb9cf48.

F-2026-16658 - Threshold signing ballots require all process participants - Low

Description: For `VoteOnTssBallot`, `votesNeeded` equals the number of participants in the active threshold signing process, requiring unanimous participation from the signing cohort. In contrast, inbound and outbound bridge-style ballots require a two-thirds majority of the eligible Universal Validator set. As a result, a single unavailable or faulty participant in the signing cohort can block progress, introducing stricter liveness requirements compared to chain-wide bridge voting.

File: `x/utss/keeper/voting.go` .

```
// votesNeeded = number of participants in the tss process
// 100% quorum needed
votesNeeded := int64(len(existing.Participants))
if votesNeeded <= 0 {
return false, false, fmt.Errorf("no participants in process %d", pr
ocessId)
}
// ...
_, isFinalized, isNew, err = k.uvalidatorKeeper.VoteOnBallot(
ctx,
ballotKey,
uvalidatortypes.BallotObservationType_BALLOT_OBSERVATION_TYPE_TSS_K
EY,
universalValidator.String(),
uvalidatortypes.VoteResult_VOTE_RESULT_SUCCESS,
existing.Participants,
int64(votesNeeded),
expiryAfterBlocks,
)
```

Operations teams must maintain high availability for every participant in an active signing process. Protocol liveness for key rotation or generation can stall on one faulty node.

Status: Accepted

Classification

Impact Rate: 2/5

Likelihood Rate: 2/5

Severity: Low

Recommendations

Remediation: After `CurrentTssProcess.ExpiryHeight`, `VoteOnTssBallot` returns an error and accepts no further votes; ballot expiry is driven by `VoteOnBallot` /

`CreateBallot` in `x/uvalidator` using the remaining blocks until that height. Recovery is starting a new process through `MsgInitiateTssKeyProcess`, which the module gates on `Params.Admin` (see `x/utss/keeper/msg_server.go`), not an arbitrary governance vote unless admin is configured that way.

Where the off-chain ceremony is configured as t-of-n (signing succeeds with t cooperating participants among n), consider setting the on-chain ballot threshold to t instead of n (`len(existing.Participants)`), so finalization aligns with the MPC layer's liveness assumptions rather than unanimity. t must be represented on-chain for the active process (stored at initiation or derived in one consistent place), `VoteOnTssBallot` must use it for `votesNeeded`, and a cryptographic review must confirm that threshold is safe for attesting the pubkey/key-id and that eligible voters still match the ceremony.

F-2026-16659 - Event-driven outbound observations set success unconditionally - Low

Description: `buildOutboundObservation` always sets `Success` to true and leaves `ErrorMsg` empty when constructing an `OutboundObservation` from event-driven processing. If a failed destination transaction were still processed through this path, validators could vote success incorrectly unless every deployment routes failures exclusively through alternate builders that set `Success` to false.

File: `push-chain-node/universalClient/chains/common/event_processor.go` .

```
observation := &uexecutortypes.OutboundObservation{
Success: true,
BlockHeight: event.BlockHeight,
TxHash: txHashHex,
ErrorMsg: "",
GasFeeUsed: gasFeeUsed,
}
```

Incorrect finalization risk arises if failure events ever reach this builder. Users could observe on-chain state that disagrees with destination-chain reality. Operators bear reputational and incident-handling cost when votes disagree with receipts.

Status: Accepted

Classification

Impact Rate: 2/5

Likelihood Rate: 2/5

Severity: Low

Recommendations

Remediation: Derive `Success` and `ErrorMsg` from receipt status, structured event fields, or verified remote procedure call data rather than constants. Guard `processOutboundEvent` so ambiguous events do not vote until success is provable. Add regression tests for reverted destination transactions and map every code path that emits `MsgVoteOutbound` for mutual exclusivity between success and failure handling.

F-2026-16868 - Fixed gas limit and fee for all vote transactions - Low

Description:

All vote operations inbound, outbound, chain metadata, TSS key process, and fund migration are submitted through the shared `vote` function, which supplies a single hard-coded gas limit, a single fee string, and fixed timeouts to `signAndBroadcastAuthZTx`. The configuration schema does not expose parameters to adjust gas, fee amount, fee denomination, or confirmation timing per deployment. If the chain's minimum fee, native denomination, or required execution gas diverges from these constants, every Universal Validator process built from this code path can fail to include or confirm vote transactions under the same conditions, which concentrates liveness and quorum risk at the application layer rather than in operator-controlled settings.

Package-level constants define gas, fee, and timing for the vote path.

`/universalClient/pushsigner/vote.go`

```
const (
defaultGasLimit = uint64(500000000)
defaultFeeAmount = "5000000000000000upc"
defaultVoteTimeout = 30 * time.Second
txPollInterval = 500 * time.Millisecond
txConfirmTimeout = 15 * time.Second
)
```

The `vote` function parses `defaultFeeAmount`, applies `defaultGasLimit` to the AuthZ transaction, and enforces `defaultVoteTimeout` for the broadcast and confirmation path; the specialized helpers `voteInbound`, `voteChainMeta`, `voteOutbound`, `voteFundMigration`, and `voteTssKeyProcess` in the same file all delegate to this implementation.

A sustained mismatch between the hard-coded fee or gas and live chain parameters can produce systematic transaction rejection or non-confirmation, which impairs voting participation for the role. Constants that substantially exceed minima can also increase fee expenditure without configuration to tune cost. Remediation for a given network currently requires a code change and release rather than configuration or runtime adjustment.

Status: Accepted

Classification

Impact Rate: 2/5

Likelihood Rate: 2/5

Severity: Low

Recommendations

Remediation: The product should allow operators to set vote gas, fee coins (amount and denomination), and relevant timeouts in `pushuv_config.json` (or equivalent), with validation against node-reported minimum gas prices and denoms where applicable. Optional integration with transaction simulation or dynamic estimation can further reduce the risk of under- or over-payment across environments.

F-2026-16874 - Coordinator retains prior validator set when refresh RPC fails - Low

Description:

The coordinator caches the universal validator set in `allValidators`. After a successful query the cache is replaced. After a failed query the function returns without modifying the cache, so the previous slice remains in memory. Coordinator selection (`IsPeerCoordinator` maps peer ID through the cached roster and compares with `coordinatorAddressForBlock`), peer and party ID resolution, and eligible-participant selection all read this cache. During membership or coordinator-period transitions on chain, repeated gRPC failures can leave the process using an outdated roster until a later refresh succeeds (including through the periodic ticker in `pollLoop`).

The refresh routine replaces the cache only after a successful RPC response; there is no invalidation branch on error.

`/universalClient/tss/coordinator/coordinator.go`

```
// updateValidators fetches and caches all validators.
func (c *Coordinator) updateValidators(ctx context.Context) {
allValidators, err := c.pushCore.GetAllUniversalValidators(ctx)
if err != nil {
c.logger.Warn().Err(err).Msg("failed to update validators cache")
return
}

c.mu.Lock()
c.allValidators = allValidators
c.mu.Unlock()

c.logger.Debug().Int("count", len(allValidators)).Msg("updated vali
dators cache")
}
```

The poll loop invokes `updateValidators` at startup and on each tick before event processing; if refresh fails on a tick, the last successful snapshot remains.

`/universalClient/tss/coordinator/coordinator.go`

```
// pollLoop polls the database for pending events and processes the
m.
func (c *Coordinator) pollLoop(ctx context.Context) {
ticker := time.NewTicker(c.pollInterval)
defer ticker.Stop()

// Update validators immediately on start
c.updateValidators(ctx)

for {
select {
case <-ctx.Done():
return
case <-c.stopCh:
return
case <-ticker.C:
// Update validators at each polling interval
```

```
c.updateValidators(ctx)
if err := c.processConfirmedEvents(ctx); err != nil {
```

Downstream logic reads `c.allValidators` under read locks; for example `IsPeerCoordinator` resolves `peerID` to `CoreValidatorAddress` from the cache and tests coordinator assignment for the current block.

Stale membership can cause incorrect coordinator identification for incoming setup messages, incorrect participant sets for new threshold rounds, or routing inconsistency relative to on-chain state until the cache reconciles. Residual risk depends on outage duration relative to validator set changes and on reliance on the next successful poll for recovery.

Status: Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 1/5

Severity: Low

Recommendations

Remediation: Invalidate or clear the cache after a failed refresh (with defined behavior when the cache is empty), enforce a maximum staleness interval after which coordinator-driven actions halt, or combine periodic refresh with backoff and alerting on repeated query failure.

Resolution: This issue has been addressed and resolved in commit 7d748bdf.

F-2026-16875 - Event confirmers advance confirmation depth without execution success - Low

Description:

On EVM chains, after a receipt is loaded, confirmation logic compares block depth to the required confirmation threshold it does not require `receipt.Status` to indicate success. On Solana, after a transaction is loaded, confirmation compares slot depth only it does not require `Meta.Err` to be absent. Events that reached the pending queue with a stored block height or slot may therefore be marked `CONFIRMED` even when the underlying transaction failed execution, and downstream voting proceeds from that record.

`/universalClient/chains/evm/event_confirmer.go`

```
requiredConfirmations := ec.getRequiredConfirmations(event.Confirma
tionType)
confirmations := latestBlock - receipt.BlockNumber.Uint64() + 1

if confirmations >= requiredConfirmations {
```

There is no branch on `receipt.Status` before the status transition.

`/universalClient/chains/svm/event_confirmer.go`

```
if confirmations >= requiredConfirmations {
rowsAffected, err := ec.chainStore.UpdateEventStatus(event.EventID,
store.StatusPending, store.StatusConfirmed)
```

There is no check of `tx.Meta.Err` prior to confirmation.

Confirmed and subsequently voted paths may include failed destination executions unless upstream indexing excludes those events from the store.

Status: Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 2/5

Severity: Low

Recommendations

Remediation: Require a successful execution outcome before promoting to `CONFIRMED`, or introduce an explicit failure path for reverted or failed transactions.

Resolution: This issue has been addressed and resolved in commit ed82cbeb.

F-2026-16876 - Observers bind to registry-supplied gateway addresses without on-chain contract integrity checks - Low

Description:

The EVM client uses `GatewayAddress` from that configuration, resolves the vault through `VAULT()` at that address, and subscribes to events using registry-supplied interface metadata. The Universal Client does not compare deployed bytecode, implementation addresses for proxies, EIP-1967 implementation slots, or code hashes to approved release artifacts.

Reliance on observing the intended gateway deployment therefore assumes correct registry state and correct RPC behavior at initialization. A defective or malicious registry change, or inappropriate reliance on an RPC endpoint during bootstrap, can direct deployments toward an unintended contract surface without an independent client-side integrity check. On Solana, the gateway program identifier is likewise taken from configuration or registry state without program-hash verification in this client.

The vault address is derived only from calls to the configured gateway address.

`/universalClient/chains/evm/client.go`

```
// Fetch vault address from gateway contract - required for event l
istening and tx building
fetchCtx, fetchCancel := context.WithTimeout(c.ctx, 15*time.Second)
vaultAddr, err := FetchVaultAddress(fetchCtx, c.rpcClient, ethcommo
n.HexToAddress(c.registryConfig.GatewayAddress))
fetchCancel()
if err != nil {
return fmt.Errorf("failed to fetch vault address from gateway: %w",
err)
}
c.logger.Info().Str("vault_address", vaultAddr.Hex()).Msg("vault ad
dress fetched from gateway")
```

No subsequent step validates deployed code against an expected hash or implementation reference.

Status: Accepted

Classification

Impact Rate: 2/5

Likelihood Rate: 1/5

Severity: Low

Recommendations

Remediation: If registry or bootstrap RPC context is wrong, validators may collectively observe and vote with respect to an unintended contract until the configuration is corrected, concentrating exposure on a single configuration path.

F-2026-16939 - Unbounded frame allocation in TSS libp2p readFramed - Low

Description:

The TSS networking layer uses a length-prefixed framing scheme on libp2p streams. The receiver reads a 32-bit length field and immediately allocates a byte slice of that size prior to reading the payload. No upper bound is enforced on the length value. A remote peer can therefore cause large, attacker-chosen heap allocations (up to approximately 4 GiB per frame, the maximum value of `uint32`) on each processed stream. This constitutes a resource-exhaustion vector against node availability. Only principals who can establish libp2p streams to the TSS listener can trigger the behavior, and deployment-specific firewalling may further restrict reachability.

`/universalClient/tss/networking/libp2p/network.go` — (`readFramed`)

```
func readFramed(r io.Reader) ([]byte, error) {
br := bufio.NewReader(r)
var length uint32
if err := binary.Read(br, binary.BigEndian, &length); err != nil {
return nil, err
}
buf := make([]byte, length)
if _, err := io.ReadFull(br, buf); err != nil {
return nil, err
}
return buf, nil
}
```

`/universalClient/tss/networking/libp2p/network.go` — (`handleStream`)

```
func (n *Network) handleStream(stream network.Stream) {
defer stream.Close()

if deadline := time.Now().Add(n.cfg.IOTimeout); true {
_ = stream.SetReadDeadline(deadline)
}

data, err := readFramed(stream)
if err != nil {
n.logger.Warn().Err(err).Msg("read failed")
return
}

n.handlerMu.RLock()
handler := n.handler
n.handlerMu.RUnlock()
if handler == nil {
return
}

// Call handler in a goroutine to avoid blocking
go handler(stream.Conn().RemotePeer().String(), data)
}
```

The libp2p host is constructed without an application-level `ConnectionGater` or analogous inbound allowlist in this module; reachability to the listener is

therefore primarily determined by network deployment rather than an explicit peer whitelist at stream open time.

`readFramed` interprets the first four octets of an inbound stream as a big-endian `uint32` frame length `L`, executes `make([]byte, L)`, and then reads exactly `L` bytes into that buffer. There is no validation that `L` is within an expected maximum for legitimate DKLS/TSS messages. The `IOTimeout` configuration bounds wall-clock read duration but does not cap allocation size: once the length prefix is consumed, the allocation has already occurred.

Status: Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 1/5

Severity: Low

Recommendations

Remediation:

1. Define a **maximum frame size** consistent with the largest legitimate TSS/DKLS message for this deployment.
2. **Reject** inbound lengths exceeding that maximum **before** allocation (return error, close stream, log at appropriate severity).
3. Optionally use `io.LimitReader` or equivalent so reads cannot exceed the cap even if the peer lies about length (combined with rejecting oversize length prefixes).
4. Apply the **same cap** to `writeFramed` for symmetry and defense in depth.

Resolution: This issue has been addressed and resolved in commit 9524142b.

F-2026-16940 - Non-idempotent ordering between failure vote submission and local persistence in the expiry sweeper - Low

Description:

The expiry sweeper processes expired, confirmed signing events by submitting a chain-side failure observation, then updating the local SQLite store to mark the event reverted. The implementation performs the on-chain vote before the `eventStore.Update` that records `StatusReverted` (and `vote_tx_hash` when applicable).

If the vote transaction is accepted by the network but the subsequent `eventStore.Update` operation fails, the local record does not reflect completion. The event therefore remains in a state returned by `GetExpiredConfirmedEvents` on subsequent sweeps. The sweep loop records the error and continues with the next event, which allows the same logical operation to be attempted again for the same `EventID`—a second failure vote or repeated observation unless the protocol module treats duplicate observations as strictly idempotent.

Outbound path

`voteOutboundFailureAndMarkReverted` invokes `pushSigner.VoteOutbound` prior to `eventStore.Update`:

```
// voteOutboundFailureAndMarkReverted submits a failure vote for an
outbound event and marks it REVERTED.
func (s *Sweeper) voteOutboundFailureAndMarkReverted(ctx context.Co
ntext, event *store.Event, errorMsg string) error {
var data uexecutortypes.OutboundCreatedEvent
if err := json.Unmarshal(event.EventData, &data); err != nil {
return fmt.Errorf("failed to parse outbound event data for event %s
: %w", event.EventID, err)
}

fields := map[string]any{"status": store.StatusReverted}

if s.pushSigner == nil {
s.logger.Warn().Str("event_id", event.EventID).Msg("pushSigner not
configured, skipping failure vote")
} else {
observation := &uexecutortypes.OutboundObservation{
Success: false,
TxHash: "",
ErrorMsg: errorMsg,
GasFeeUsed: "0",
}
voteTxHash, err := s.pushSigner.VoteOutbound(ctx, data.TxID, data.U
niversalTxId, observation)
if err != nil {
return fmt.Errorf("failed to vote failure for event %s: %w", event.
EventID, err)
}
fields["vote_tx_hash"] = voteTxHash
}

if err := s.eventStore.Update(event.EventID, fields); err != nil {
return fmt.Errorf("failed to mark event %s as reverted: %w", event.
EventID, err)
}
s.logger.Info().
Str("event_id", event.EventID).
Str("tx_id", data.TxID).
```

```
Str("error_msg", errorMsg).
Msg("voted outbound failure and marked REVERTED")
return nil
}
```

Fund migration path

`voteFundMigrationFailureAndMarkReverted` applies the same ordering:

```
// voteFundMigrationFailureAndMarkReverted submits a failure vote f
or a fund migration event and marks it REVERTED.
func (s *Sweeper) voteFundMigrationFailureAndMarkReverted(ctx conte
xt.Context, event *store.Event, errorMsg string) error {
var data utsstypes.FundMigrationInitiatedEventData
if err := json.Unmarshal(event.EventData, &data); err != nil {
return fmt.Errorf("failed to parse fund migration event data for ev
ent %s: %w", event.EventID, err)
}

fields := map[string]any{"status": store.StatusReverted}

if s.pushSigner == nil {
s.logger.Warn().Str("event_id", event.EventID).Msg("pushSigner not
configured, skipping failure vote")
} else {
voteTxHash, err := s.pushSigner.VoteFundMigration(ctx, data.Migrati
onID, "", false)
if err != nil {
return fmt.Errorf("failed to vote fund migration failure for event
%s: %w", event.EventID, err)
}
fields["vote_tx_hash"] = voteTxHash
}

if err := s.eventStore.Update(event.EventID, fields); err != nil {
return fmt.Errorf("failed to mark event %s as reverted: %w", event.
EventID, err)
}
s.logger.Info().
Str("event_id", event.EventID).
Uint64("migration_id", data.MigrationID).
Str("error_msg", errorMsg).
Msg("voted fund migration failure and marked REVERTED")
return nil
}
```

Control flow on persistence failure

Per-event failures in the sweep iteration do not halt the sweep; processing advances via `continue`, which permits re-selection of the same event until local state converges:

```
for _, event := range events {
if event.Type == store.EventTypeSignOutbound {
if err := s.voteOutboundFailureAndMarkReverted(ctx, &event, "event
expired before TSS could start"); err != nil {
s.logger.Error().Err(err).Str("event_id", event.EventID).Msg("faile
d to sweep expired SIGN_OUTBOUND event")
continue
}
} else if event.Type == store.EventTypeSignFundMigrate {
if err := s.voteFundMigrationFailureAndMarkReverted(ctx, &event, "e
vent expired before TSS could start"); err != nil {
s.logger.Error().Err(err).Str("event_id", event.EventID).Msg("faile
d to sweep expired SIGN_FUND_MIGRATE event")
continue
}
} else {
if err := s.eventStore.Update(event.EventID, map[string]any{"status
": store.StatusReverted}); err != nil {
s.logger.Error().Err(err).Str("event_id", event.EventID).Msg("faile
d to revert expired key event")
continue
```

```
}
}
swept++
}
```

Where the client does not suppress duplicate submissions before broadcast, repeated failure votes incur additional transaction fees and consume account sequence. The substantive effect on ledger state and operator workload depends on how `x/uexecutor` and `x/utss` treat duplicate failure observations: they may be treated as idempotent no-ops, rejected at execution with recurring errors, or reflected in higher log volume. Any consequence for validator or operator standing is defined by those modules and should be validated against the live protocol rules.

Status: Accepted

Classification

Impact Rate: 2/5

Likelihood Rate: 1/5

Severity: Low

Recommendations

Remediation: The ordering hazard is addressed when local persistence and chain submission share a single, replay-safe lifecycle: failure processing remains eligible for resweep only until durable evidence exists that the vote will not be duplicated (for example, an intermediate status, persisted `vote_tx_hash`, or vote intent recorded atomically with exclusion from the expired-confirmed query set). Retry policy after broadcast ought to gate on inclusion or on-chain reconciliation rather than immediate resubmission. A preparatory local flag—where permitted by module semantics—may separate “vote pending” from `StatusReverted` until both submission and storage succeed; recovery logic must then treat an accepted vote as terminal for that event regardless of subsequent database errors.

F-2026-16944 - Event Skipping via Pagination Failure in Solana Event Listener - Low

Description:

The `processSlotRange` function in the SVM event listener fails to implement pagination when fetching transaction signatures, leading to permanent data loss and missed cross-chain events during periods of high network activity.

The function located in `./push-chain-node-clean/universalClient/chains/svm/event_listener.go` is responsible for discovering and processing inbound events from the Solana gateway address within a specific slot range.

The implementation relies on the `GetSignaturesForAddress` method, which wraps the Solana RPC call `getSignaturesForAddress`. Per the Solana RPC specification, this endpoint is paginated and returns a maximum of 1,000 signatures per request. To retrieve the full history of transactions, a client must iteratively call the endpoint using the `before` parameter, passing the signature of the oldest transaction from the previous batch.

The current implementation performs a single call to `GetSignaturesForAddress` without any pagination logic:

```
signatures, err := el.rpcClient.GetSignaturesForAddress(ctx, gatewa
yAddr)
// ... filtering logic follows ...
```

Because the code does not loop to fetch subsequent pages, it only retrieves the 1,000 most recent signatures. If more than 1,000 transactions are sent to the gateway address between polling intervals, any transactions exceeding this limit, even those falling within the target `fromSlot` and `toSlot` range, will be ignored.

Status: Fixed

Classification

Impact Rate: 4/5

Likelihood Rate: 1/5

Severity: Low

Recommendations

Remediation: Update the `processSlotRange` function to implement a pagination loop.
Note: Implement reasonable limits in the number of events to prevent DoS via unbounded event processing.

Resolution: This issue has been addressed and resolved in commit 35bd9b67.

F-2026-16964 - [PUSHCHAIN REPORTED] Issue 5 — SVM TX resolver lacks a way to differentiate invalid signature vs transient simulation failure - Low

Description:

Description

Currently tx may be marked reverted if CPI reverts, but this can be transient (market conditions, account state changes). Without a signature validity check, false reverts can lead to replay/double-spend style issues.

Status: Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 2/5

Severity: Low

Recommendations

Remediation: Add `deadline` to the TSS-signed SVM payload and enforce `Clock::unix_timestamp <= deadline` in gateway TSS verification alongside existing signature and `ExecutedSubTx` replay checks. Keep PDA-based execution detection for the multi-relayer race. Align Push outbound failure/revert voting with the deadline window. Coordinate the SVM program, UV client, and TSS rollout document deadline source and duration.

Resolution: This issue has been addressed and resolved in commit fcb0f786.

F-2026-16965 - [PUSHCHAIN REPORTED] Issue 6 — Reverted txs can get stuck due to architecture (failure visibility limited to signer set) - Low

Description: Successful outbounds are observed by every UV and voted accordingly. Failed txs (revert/other failure) are primarily visible to the signing set (2/3). If a signing node crashes, it might never mark the tx as reverted, leaving funds stuck as pending.

There is also an expiry sweeper that marks tx reverted after N time; but that can cause multiple signatures, double spend risks, and operational confusion.

Status: Fixed

Classification

Impact Rate: 4/5

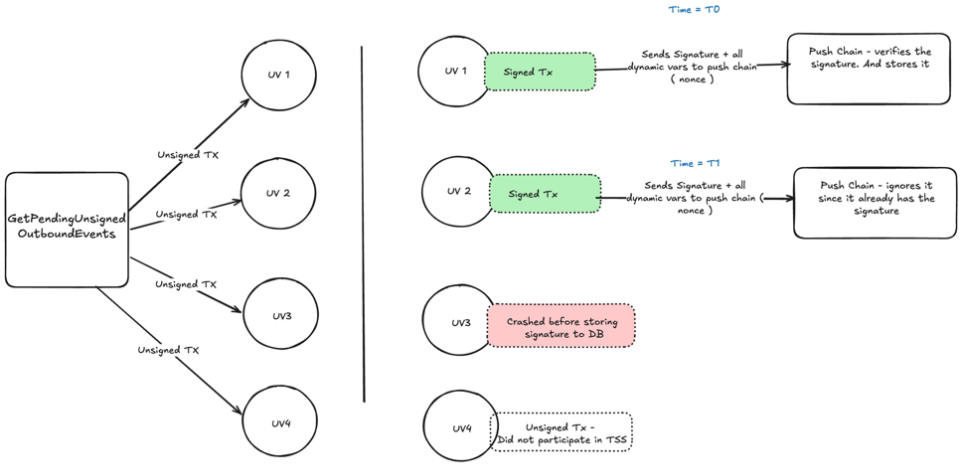
Likelihood Rate: 1/5

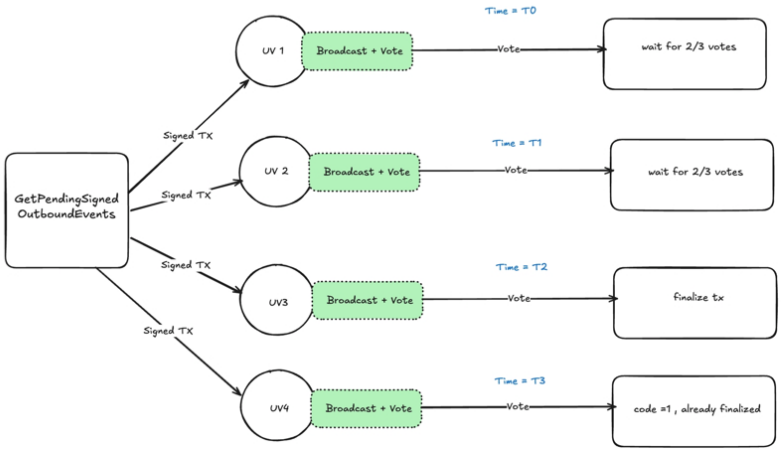
Severity: Low

Recommendations

Remediation: Draft approach (TBD)

- All the signing participants should report signature which should be verified by push chain
- Later all the UVs can try to broadcast and process these tx and vote accordingly.





Resolution: This issue has been addressed and resolved in commits [83a05289](#) and [1fa5a610](#).

`83a05289` — After TSS signing, signatures are broadcast to all UVs via `signature_broadcast`; each node verifies and persists locally so non-signers can also broadcast and resolve reverts. Coordinator setup/ACK paths accept existing signatures for crash recovery; signing deadlines and EVM/SVM revert/broadcaster handling were tightened.

`1fa5a610` — Hardens crash recovery: `RecoverInProgressEvents()` promotes IN_PROGRESS rows with persisted `signing_data` to SIGNED on startup, session expiry restores SIGNED when a signature exists, and Solana tx resolution is fixed so all UVs can settle in-progress outbounds.

F-2026-16967 - [PUSHCHAIN-REPORTED] Issue 8 - MsgExecutePayload rejects when precomputed UEA is pre-funded but not deployed - Low

Description:

Background -current deployment model

UEA addresses on Push Chain are deterministic (Ref) (CREATE2 salt = keccak256(chainNamespace, chainId, owner)), so they can be precomputed before deployment. A UEA can only be deployed through the inbound flow:

1. User on external chain calls UniversalGateway.sendUniversalTx(...) (any of the entry methods at UniversalGateway.sol:306,314,341).
2. UVs observe the UniversalTx event, vote `MsgVoteInbound`.
3. After 2/3+ quorum, x/uexecutor finalizes the inbound and deploys the UEA as part of inbound execution if the UEA isn't deployed yet.
4. From that point on, the user can submit gasless MsgExecutePayload with an EIP-712-signed UniversalPayload. Anyone (relayer, paymaster) can submit the message; the UEA contract verifies the signature against universalAccountId.Owner.

There is no user-callable MsgDeployUEA. The inbound is the only bootstrap path, and MsgExecutePayload rejected with "UEA is not deployed" if the UEA didn't exist yet.

Description

The proposed change lets a user deploy their UEA and use the gasless MsgExecutePayload route even if someone tries to sabotage them by

pre-sending funds to the deterministic UEA address. Without it, a third party can dust-transfer to a user's precomputed UEA address, and a balance-based SDK that decides between "deploy via inbound" vs "execute via gasless MsgExecutePayload" by reading the on-chain balance routes the user into the gasless path — where the handler rejects with "UEA is not deployed". The user can still use their UEA, but only by performing a fresh inbound (which costs source-chain gas and bridging fees) — asymmetric griefing on the user's first interaction.

The auto-deploy is gated on the precomputed UEA address holding a non-zero native balance. This gate is also what prevents the gasless handler from being a DDoS vector: an unfunded UEA cannot successfully execute a payload anyway (gas-fee deduction at the end of MsgExecutePayload would fail and the tx would roll back), so deploying-on-demand for unfunded addresses would just be wasted work. By requiring pre-funding,

we filter out the class of requests that are doomed to fail and only auto-deploy for addresses that can actually pay for their own execution.

Status: Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 2/5

Severity: Low

Recommendations

Remediation: Allow MsgExecutePayload to auto-deploy the UEA inline when the precomputed UEA address holds a non-zero native balance. Concretely, in msg_execute_payload.go:

```
if !isDeployed {
// only auto-deploy if the precomputed UEA address has been funded
ueaAccAddr := sdk.AccAddress(ueaAddr.Bytes())
balance := k.bankKeeper.GetBalance(sdkCtx, ueaAccAddr, pchaintypes.
BaseDenom)
if balance.Amount.Sign() == 0 {
return fmt.Errorf("UEA is not deployed")
}
if _, err := k.DeployUEAV2(ctx, evmFrom, universalAccountId); err !
= nil {
return errors.Wrapf(err, "failed to auto-deploy pre-funded UEA")
}
}
```

Resolution: This issue has been addressed and resolved in commit 1876435.

F-2026-16991 - Stale active universal validators inflate ballot quorum and can deadlock finalization - Low

Description:

Eligible voters for cross-chain ballots are taken from the universal validator set by lifecycle status only entries in `UV_STATUS_ACTIVE` or `UV_STATUS_PENDING_JOIN` are counted. The enumerator does not require that the corresponding Cosmos staking validator still be bonded or that slashing has not tombstoned the operator. The module is not registered as a hook on `StakingKeeper`, so jailing, unbonding, slashing, and tombstone events on the base chain do not automatically clear or downgrade universal-validator rows. Administrators must remove or update stranded entries explicitly.

Ballot creation in the executor derives the two-thirds threshold from the length of that eligible list and passes the threshold into `uvalidator` where it is stored on the ballot. Vote admission in the executor correctly rejects signers who are not bonded or who are tombstoned, but the frozen denominator still counts those addresses. When enough universal validators remain `ACTIVE` on paper while no longer being able to vote in practice, quorum becomes impossible. Ballots use a very large default expiry horizon, so stuck ballots do not age out on a human timescale, and thresholds on already-created ballots are not recomputed when the set is repaired later.

File: `push-chain-node/x/uvalidator/keeper/validator.go` .

```
// GetEligibleVoters returns all validators that are eligible to vo
te on external transactions.
// Eligibility: validators with status ACTIVE or PENDING_JOIN.
func (k Keeper) GetEligibleVoters(ctx context.Context) ([]types.Uni
versalValidator, error) {
var voters []types.UniversalValidator

err := k.UniversalValidatorSet.Walk(ctx, nil, func(addr sdk.ValAddr
ess, val types.UniversalValidator) (stop bool, err error) {
switch val.LifecycleInfo.CurrentStatus {
case types.UVStatus_UV_STATUS_ACTIVE, types.UVStatus_UV_STATUS_PEND
ING_JOIN:
voters = append(voters, val)
}
return false, nil
})
```

File: `push-chain-node/x/uexecutor/keeper/voting.go`

```
universalValidatorSet, err := k.uvalidatorKeeper.GetEligibleVoters(
ctx)
if err != nil {
return false, false, err
}

totalValidators := len(universalValidatorSet)

votesNeeded := (types.VotesThresholdNumerator*totalValidators)/type
s.VotesThresholdDenominator + 1
// ...
```

```
_, isFinalized, isNew, err = k.uvalidatorKeeper.VoteOnBallot(
ctx,
ballotKey,
uvalidatortypes.BallotObservationType_BALLOT_OBSERVATION_TYPE_INBOU
ND_TX,
universalValidator.String(),
uvalidatortypes.VoteResult_VOTE_RESULT_SUCCESS,
universalValidatorSetStrs,
int64(votesNeeded),
int64(types.DefaultExpiryAfterBlocks),
)
```

File: `push-chain-node/x/uvalidator/types/ballot.go`

```
func NewBallot(
id string,
ballotType BallotObservationType,
voters []string,
votesNeeded int64,
createdHeight int64,
expiryAfterBlocks int64,
) Ballot {
b := Ballot{
Id: id,
BallotType: ballotType,
EligibleVoters: voters,
VotingThreshold: votesNeeded,
Status: BallotStatus_BALLOT_STATUS_PENDING,
BlockHeightCreated: createdHeight,
BlockHeightExpiry: createdHeight + expiryAfterBlocks,
}
b.InitEmptyVotes()
return b
}
```

File: `push-chain-node/x/uexecutor/keeper/msg_server.go`

```
isBonded, err := ms.k.uvalidatorKeeper.IsBondedUniversalValidator(c
tx, msg.Signer)
// ...
if !isBonded {
return nil, fmt.Errorf("universal validator for signer %s is not bo
nded", msg.Signer)
}

isTombstoned, err := ms.k.uvalidatorKeeper.IsTombstonedUniversalVal
idator(ctx, msg.Signer)
// ...
if isTombstoned {
return nil, fmt.Errorf("universal validator for signer %s is tombst
oned", msg.Signer)
}
```

File: `push-chain-node/app/app.go`

```
// register the staking hooks
app.StakingKeeper.SetHooks(
stakingtypes.NewMultiStakingHooks(
app.DistrKeeper.Hooks(),
app.SlashingKeeper.Hooks(),
),
)
```

No `x/uvalidator` staking hook is registered here, so base-chain lifecycle events do not drive universal-validator reconciliation.

File: `push-chain-node/x/uexecutor/types/constants.go`

```
// Default number of blocks after which ballot expires.
// Set to 100M (~19 years at 6s blocks) to effectively disable expi
ry.
DefaultExpiryAfterBlocks = 100_000_000
```

Let `N` denote the count returned by `GetEligibleVoters` at ballot creation and `S` the count of those entries that can no longer cast valid votes because they are unbonded, jailed, or tombstoned on staking. The ballot stores a threshold based on `N` while live votes are capped by `N - S`. When `S` is large enough, passing the stored threshold is impossible, so inbound and outbound observations stop finalizing. Because expiry is configured for an extremely long horizon and the threshold is fixed at creation, cleaning the universal validator set afterward does not retroactively fix ballots that were already created. Users experience stuck deposits or settlements operators lose a clear on-chain escape hatch unless governance or code adds one.

Status: Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 1/5

Severity: Low

Recommendations

Remediation:

- Subscribe to staking and slashing hooks (or an equivalent periodic reconciler) so universal-validator lifecycle moves out of `ACTIVE` when the base validator unbonds, is jailed, or is tombstoned, and coordinate with executor and TSS hooks as needed.
- Narrow `GetEligibleVoters` so the denominator matches who may vote, for example by requiring bonded and non-tombstoned staking state before including a row.
- Provide a governed or operational path to recompute `EligibleVoters` and `VotingThreshold` on existing pending ballots when the set was wrong, or to cancel and recreate ballots with correct parameters.

Resolution: This issue has been addressed and resolved in commit 38fbf73.

F-2026-16992 - Removing an active universal validator bypasses the ongoing-TSS guard used elsewhere - Low

Description:

When an administrator updates universal-validator status through the dedicated status message, the keeper refuses the change if the TSS module reports an ongoing process. When an administrator removes a validator that is currently `ACTIVE`, the removal path transitions the validator to `pending leave` without calling that same guard. The pending-join branch does consult current TSS participants before allowing removal, so protection is asymmetric: one admin path respects in-flight key material while another can still mutate the eligible set during keygen or resharing.

Removing an active validator triggers hooks that lead `x/utss` to start a new key process. That routine force-expires any still-active TSS process by setting its expiry height into the past, then begins a new round. A mistaken or malicious removal during TSS therefore tears down the in-flight round and can strand or confuse off-chain participants until operators recover manually.

File: `push-chain-node/x/uvalidator/keeper/msg_update_universal_validator_status.go`

```
isOngoingTSS, err := k.UtssKeeper.HasOngoingTss(ctx)
if err != nil {
return fmt.Errorf("failed to check TSS state: %w", err)
}
if isOngoingTSS {
k.Logger().Warn("cannot update validator status: TSS process is ong
oing",
"validator", coreValidatorAddr,
"requested_status", newStatus.String(),
)
return fmt.Errorf("cannot update validator status: TSS process is o
ngoing")
}
```

File: `push-chain-node/x/uvalidator/keeper/msg_remove_universal_validator.go`

```
switch val.LifecycleInfo.CurrentStatus {
case types.UVStatus_UV_STATUS_ACTIVE:
// Active -> Pending Leave
if err := k.UpdateValidatorStatus(ctx, valAddr, types.UVStatus_UV_S
TATUS_PENDING_LEAVE); err != nil {
return fmt.Errorf("failed to mark validator %s as pending leave: %w
", universalValidatorAddr, err)
}

newStatus = types.UVStatus_UV_STATUS_PENDING_LEAVE

case types.UVStatus_UV_STATUS_PENDING_JOIN:
// Check if validator is part of the current TSS process
currentParticipants, err := k.UtssKeeper.GetCurrentTssParticipants(
ctx)
// ...
if isParticipant {
return fmt.Errorf("validator %s is part of the current TSS process
and cannot be removed", universalValidatorAddr)
}
```

The `ACTIVE` branch contains no `HasOngoingTss` or participant check before `UpdateValidatorStatus`.

File: `push-chain-node/x/utss/keeper/initiate_tss_key_process.go`

```
existing, err := k.CurrentTssProcess.Get(ctx)
if err == nil {
if currentHeight < existing.ExpiryHeight {
k.Logger().Info("Validator set changed: force-expiring current TSS
process",
"old_process_id", existing.Id,
"old_expiry", existing.ExpiryHeight,
"current_height", currentHeight)

existing.ExpiryHeight = currentHeight - 1
// ...
}
}
```

An admin message that moves an `ACTIVE` universal validator to `pending leave` during TSS cancels the current process in favor of a new one without the same explicit refusal that status updates receive. Operators who assume "the chain blocks UV changes during TSS" may be misled because that rule does not apply to removal. Under a compromised admin key, the gap becomes an intentional denial vector against keygen. Even in benign cases, accidental removal mid-round wastes work and requires coordinated restart of off-chain signers.

Status: Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 1/5

Severity: Low

Recommendations

Remediation:

- Reuse `HasOngoingTss` at the start of `RemoveUniversalValidator`, or before the `ACTIVE` branch, mirroring `UpdateUniversalValidatorStatus`.
- Alternatively, reject removal of any address returned by `GetCurrentTssParticipants` regardless of universal-validator lifecycle state, so active participants cannot be dropped during a round.
- Document until fixed that `MsgRemoveUniversalValidator` is not safe to send while TSS is in progress.
- Consider surfacing a single policy module or precondition helper shared by all admin mutators of the universal validator set.

Resolution: This issue has been addressed and resolved in commit 3f3d73d.

F-2026-16993 - Votes are accepted on pending ballots without checking block height expiry - Low

Description: Ballots carry an expiry height computed at creation. Expiry is applied when new ballots are created, because `CreateBallot` runs `ExpireBallotsBeforeHeight` first. Existing ballots are returned unchanged from `GetOrCreateBallot`, and the vote handler only requires status `PENDING`. Neither path compares the current block height to `BlockHeightExpiry` before recording a vote. Therefore a ballot that has passed its nominal expiry but has not yet been marked `EXPIRED` in state (because no subsequent `CreateBallot` ran to trigger the sweep) can still accept votes and reach a passing finalization. Outcomes can depend on whether another ballot creation occurred in between, which is a poor contract for operators and for any logic that assumes expiry is a hard cutoff at the vote boundary.

File: `push-chain-node/x/uvalidator/keeper/voting.go` ·

```
ballot, isNew, err = k.GetOrCreateBallot(ctx, id, ballotType, voter
s, votesNeeded, expiryAfterBlocks)
if err != nil {
return ballot, false, false, errors.Wrap(err, "Error while voting o
n the ballot")
}

if ballot.Status != types.BallotStatus_BALLOT_STATUS_PENDING {
k.Logger().Warn("ballot is not in pending state, cannot vote",
"ballot_id", id,
"ballot_status", ballot.Status.String(),
"voter", voter,
)
return ballot, false, false, fmt.Errorf("ballot %s is already %s",
id, ballot.Status.String())
}
```

There is no `ballot.IsExpired(currentHeight)` check in this flow.

File: `push-chain-node/x/uvalidator/types/ballot.go` ·

```
func (b Ballot) AddVote(address string, vote VoteResult) (Ballot, e
rror) {
if b.Status != BallotStatus_BALLOT_STATUS_PENDING {
return b, fmt.Errorf("cannot vote on ballot %s: not pending", b.Id)
}
// ...
}

func (b Ballot) IsExpired(currentHeight int64) bool {
return currentHeight >= b.BlockHeightExpiry
}
```

File: `push-chain-node/x/uvalidator/keeper/ballot.go` ·

```
if err := k.ExpireBallotsBeforeHeight(ctx, blockHeight); err != nil
{
return types.Ballot{}, err
}
```

```
func (k Keeper) GetOrCreateBallot(
ctx context.Context,
id string,
ballotType types.BallotObservationType,
voters []string,
votesNeeded int64,
expiryAfterBlocks int64,
) (types.Ballot, bool, error) {

if ballot, err := k.Ballots.Get(ctx, id); err == nil {
k.Logger().Debug("ballot found (existing)", "ballot_id", id)
return ballot, false, nil
}

newBallot, err := k.CreateBallot(ctx, id, ballotType, voters, votes
Needed, expiryAfterBlocks)
```

Returning an existing ballot skips `ExpireBallotsBeforeHeight`.

File: `push-chain-node/x/uvalidator/abci.go` ·

```
func BeginBlocker(ctx sdk.Context, uvalidatorKeeper keeper.Keeper)
error {
// ...
if height > 1 {
if err := AllocateTokens(ctx, previousTotalPower, ctx.VoteInfos(),
uvalidatorKeeper); err != nil {
```

`BeginBlocker` allocates tokens and does not call `ExpireBallotsBeforeHeight`.

Votes cast after the documented expiry height can still finalize a ballot as passed if the lazy expiry sweep has not run. Downstream modules cannot treat `BlockHeightExpiry` as a strict cutoff unless they duplicate height checks. When traffic is low, the order in which unrelated ballots are created can change whether a late vote succeeds or fails after another creation triggers expiry, which is confusing for monitoring and incident response. Today the default expiry span from the executor is extremely large, so the defect is mostly latent until expiry parameters become operationally meaningful.

Status: Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Severity: Low

Recommendations

Remediation:

- Reject votes in `VoteOnBallot` (or inside vote aggregation) when `ballot.IsExpired(currentHeight)`, transition the ballot to `EXPIRED`, and move identifiers between active and expired collections for consistency.

- Invoke `ExpireBallotsBeforeHeight` from `BeginBlocker` or `EndBlocker` each height so state tracks expiry independently of ballot creation rate.
- Ensure finalization logic refuses to mark a ballot passed when the current height is at or beyond `BlockHeightExpiry`, even if the yes-count threshold is met afterward.

Resolution: This issue has been addressed and resolved in commit 3948c36.

F-2026-17022 - Case-sensitive token keys versus case-insensitive PRC20 lookup and EIP-55 observer addresses - Low

Description:

Token configuration entries are stored under a composite key built from the chain identifier and the token address string with whitespace trimmed but case preserved. Direct reads use that key verbatim, while `GetTokenConfigByPRC20` compares PRC20 contract addresses using lowercasing. The EVM observer populates inbound asset addresses using `go-ethereum`'s `Address.Hex()`, which yields EIP-55 mixed-case strings. An administrator who registers the same logical token using all-lowercase text from a block explorer therefore stores a different key than the one the executor looks up at runtime, so inbounds can fail even though a token row exists. Two registrations that differ only in case are treated as distinct rows, which invites duplicate configuration and asymmetric behavior between inbound and outbound resolution paths.

File: `push-chain-node/x/uregistry/types/keys.go`

```
func GetTokenConfigsStorageKey(chain, address string) string {
return fmt.Sprintf("%s:%s", chain, strings.TrimSpace(address))
}
```

File: `push-chain-node/x/uregistry/keeper/keeper.go`

```
func (k Keeper) GetTokenConfig(ctx context.Context, chain, address
string) (types.TokenConfig, error) {
storageKey := types.GetTokenConfigsStorageKey(chain, address)
config, err := k.TokenConfigs.Get(ctx, storageKey)
if err != nil {
return types.TokenConfig{}, err
}
return config, nil
}
```

File: `push-chain-node/x/uregistry/keeper/keeper.go`

```
func (k Keeper) GetTokenConfigByPRC20(
ctx context.Context,
chain string,
prc20Addr string,
) (types.TokenConfig, error) {

prc20Addr = strings.ToLower(strings.TrimSpace(prc20Addr))

var found *types.TokenConfig

err := k.TokenConfigs.Walk(ctx, nil, func(
key string,
cfg types.TokenConfig,
) (bool, error) {

if cfg.Chain != chain {
return false, nil
}

if cfg.NativeRepresentation == nil {
return false, nil
}
```

```
if strings.ToLower(cfg.NativeRepresentation.ContractAddress) == prc
20Addr {
found = &cfg
return true, nil
}

return false, nil
})
```

File: `push-chain-node/universalClient/chains/evm/event_parser.go`

```
payload.Token = ethcommon.BytesToAddress(log.Data[0*32+12 : 0*32+32
]).Hex()
payload.Amount = new(big.Int).SetBytes(log.Data[1*32 : 2*32]).Strin
g()
```

Downstream, `x/uexecutor` resolves inbound assets via `GetTokenConfig` using that string as the address component, so any mismatch between registered casing and observed casing produces a not-found result and revert-style handling on the inbound path.

Operators can see systematic inbound failures after a token was "registered," because the stored key does not match the checksum form emitted by the client. Outbound flows that rely on PRC20 lookup may still resolve, so the two directions disagree. Duplicate rows for the same token under different casings create ambiguous or non-deterministic behavior when walking the map, and removals keyed by the wrong case appear to succeed at the message layer while leaving stale entries in state.

Status: Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Severity: Low

Recommendations

Remediation:

- Canonicalize EVM addresses when forming storage keys (for example lowercase for `eip155:*` namespaces) and apply the same rule on read and remove paths.
- Enforce valid hex address form and a single canonical representation in `TokenConfig.ValidateBasic` (and related fields such as native representation contract addresses).
- Reject duplicate registrations for the same chain when addresses are equal under case-insensitive comparison.
- Add a one-time migration to rewrite existing keys to the canonical form where mixed-case keys already exist.

Resolution: This issue has been addressed and resolved in commit 93de525.

F-2026-17025 - isContractDeployed treats any account with a non-empty code hash as a deployed contract - Low

Description:

Before deploying each system proxy, the registry checks whether the proxy address already has a “deployed” contract. The implementation treats any account whose `CodeHash` slice is non-empty as deployed. In Ethereum-compatible state, externally owned accounts still carry the keccak256 hash of empty bytecode (a fixed 32-byte sentinel), so the length check is true for EOAs as well as for contracts. On a fresh genesis with no prior accounts at the reserved addresses, deployment proceeds as intended. On a chain where an address was touched as an EOA before deployment runs, the predicate can skip the entire deploy sequence for that slot, leaving system contracts missing while logs suggest they were already present.

File: `push-chain-node/x/uregistry/keeper/genesis.go` ·

```
func isContractDeployed(
ctx sdk.Context,
evmKeeper types.EVMKeeper,
addr common.Address,
) bool {
acc := evmKeeper.GetAccount(ctx, addr)
return acc != nil && acc.CodeHash != nil && len(acc.CodeHash) != 0
}
```

File: `push-chain-node/x/uregistry/keeper/genesis.go` ·

```
proxyAddr := common.HexToAddress(contract.Address)
if isContractDeployed(sdkCtx, evmKeeper, proxyAddr) {
sdkCtx.Logger().Info(
"system contract already deployed, skipping",
"name", name,
"proxy", proxyAddr.Hex(),
)
continue
}
```

The skip branch omits proxy admin, implementation, and proxy deployment for that name.

File: `push-chain-node/x/uexecutor/types/constants.go` ·

```
// EmptyCodeHash is the keccak256 hash of empty bytes — the code ha
sh for EOAs (no contract code).
var EmptyCodeHash = common.HexToHash("0xc5d2460186f7233c927e7db2dcc
703c0e500b653ca82273b7bfad8045d85a470")
```

File: `push-chain-node/x/uexecutor/keeper/execute_inbound_gas_and_payload.go` ·

```
codeHash := k.evmKeeper.GetCodeHash(sdkCtx, ueaAddr)
if codeHash != types.EmptyCodeHash && codeHash != (common.Hash{}) {
```

```
    isSmartContract = true
}
```

A future upgrade that introduces a new reserved address or re-runs deployment on live state could skip installation if anyone created an EOA or sent value to that address first. The chain would then lack the expected proxy triple for that system contract, with only an informational log as the signal. Manual state surgery or a patched deploy routine would be required to recover.

Status: Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 1/5

Severity: Low

Recommendations

Remediation:

- Replace the predicate with an explicit comparison against the empty-code-hash sentinel (and zero hash if applicable), using `GetCodeHash` or `BytesToHash` on `CodeHash`, consistent with `x/uexecutor`.
- Centralize `EmptyCodeHash` in a shared types package or import the existing constant so both modules agree.
- Add a test that seeds an EOA at a system proxy address and asserts that deployment still runs or that behavior matches the chosen policy.
- If reserved addresses are protocol-owned regardless of prior activity, document that policy and consider overwrite semantics for those slots instead of skip-on-any-account.

Resolution: This issue has been addressed and resolved in commit a426e3f.

F-2026-17038 - Genesis export/import omits pending TSS event index - Low

Description:

The `x/utss` module keeps a secondary index `PendingTssEvents` (`process_id` → `event_id`) for active process-initiated events. `GenesisState` in protobuf does not include this map, and `InitGenesis` never rebuilds it from the restored `tss_events` slice. After genesis export and import (or a handcrafted genesis that lists events but not the index), RPC queries that read `PendingTssEvents` return errors or empty results even when matching `TssEvent` rows exist.

Genesis schema has no pending index field — `push-chain-node/proto/utss/v1/genesis.proto`

```
// GenesisState defines the module genesis state
message GenesisState {
// Params defines all the parameters of the module.
Params params = 1 [(gogoproto.nullable) = false];
// ...
// tss_events are entries from the TssEvents store.
repeated TssEvent tss_events = 7 [(gogoproto.nullable) = false];
// next_tss_event_id is the next auto-increment ID for TssEvents.
uint64 next_tss_event_id = 8;
// ...
}
```

Init restores events but not `**PendingTssEvents**` — `push-chain-node/x/utss/keeper/keeper.go`

```
// Restore TssEvents
for _, event := range data.TssEvents {
if err := k.TssEvents.Set(ctx, event.Id, event); err != nil {
return err
}
}

// Restore NextTssEventId
if data.NextTssEventId > 0 {
if err := k.NextTssEventId.Set(ctx, data.NextTssEventId); err != ni
l {
return err
}
}
```

There is no loop calling `PendingTssEvents.Set` for events with `TSS_EVENT_PROCESS_INITIATED` and `TSS_EVENT_ACTIVE` (or equivalent pending semantics).

Queries depend on the index — `push-chain-node/x/utss/keeper/query_server.go`

```
func (k Querier) GetPendingTssEvent(goCtx context.Context, req *typ
es.QueryGetPendingTssEventRequest) (*types.QueryGetPendingTssEventR
esponse, error) {
ctx := sdk.UnwrapSDKContext(goCtx)

eventId, err := k.Keeper.PendingTssEvents.Get(ctx, req.ProcessId)
if err != nil {
```

```
return nil, err
}
// ...
}
```

Operators and the `universalClient/tss` stack that rely on pending-event queries can lose visibility into the active TSS process immediately after a genesis-based restart, even though `AllTssEvents` still lists the underlying rows. That complicates recovery, monitoring, and automation that keys off the pending index.

Status: Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 1/5

Severity: Low

Recommendations

Remediation:

1. **Preferred:** After loading `TssEvents` in `InitGenesis`, rebuild `PendingTssEvents` deterministically (for each event matching pending semantics, `Set(process_id, event_id)`), and assert uniqueness invariants.
2. **Alternative:** Extend `GenesisState` with an explicit repeated pending-index entry and validate it against `tss_events` on import.
3. Add genesis tests that export state with an active pending process-initiated event, re-import, and assert `GetPendingTssEvent` succeeds.

Resolution: This issue has been addressed and resolved in commit 7be2938.

F-2026-17041 - Fund migration ballot key uses raw txHash string - Low

Description:

Fund migration voting derives ballot keys from `migrationId`, the raw `txHash` string, and the boolean `success`. There is no canonicalization (for example lowercasing hex, stripping `0x`) in `x/utss` before hashing. The same underlying chain transaction can therefore produce multiple distinct ballots, fragmenting quorum.

Ballot key includes raw `**txHash**`

`push-chain-node/x/utss/types/keys.go`

```
func GetFundMigrationBallotKey(migrationId uint64, txHash string, s
uccess bool) string {
canonical := fmt.Sprintf("fm:%d:%s:%t", migrationId, txHash, succes
s)
h := sha256.Sum256([]byte(canonical))
return hex.EncodeToString(h[:])
}
```

Vote path passes message hash through unchanged

`push-chain-node/x/utss/keeper/voting.go`

```
func (k Keeper) VoteOnFundMigrationBallot(
ctx context.Context,
universalValidator sdk.ValAddress,
migrationId uint64,
txHash string,
success bool,
) (isFinalized bool, isNew bool, err error) {

ballotKey := types.GetFundMigrationBallotKey(migrationId, txHash, s
uccess)
```

`MsgVoteFundMigration` has no `ValidateBasic` in `x/utss/types` that normalizes `txHash` before the message server calls `VoteFundMigration`.

Without a single canonical representation, honest validators observing the same migration transaction can fail to aggregate votes, leaving migrations pending until operators converge on an off-chain convention or retry with a normalized hash.

Status: Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 1/5

Severity: Low

Recommendations

Remediation:

1. Canonicalize `txHash` once in the message server (or in `VoteFundMigration`) using the same rules as other observation paths (strip `0x`, lowercase for EVM-length hex, reject malformed lengths).
2. Add regression tests with equivalent hash encodings that must map to one ballot.

Resolution: This issue has been addressed and resolved in commit 93de525.

F-2026-17043 - usigverifier Ed25519 precompile verifies signature over ASCII hex, not the raw message - Low

Description:

`verifyEd25519` takes a `bytes32` value from the caller but checks the signature against the **characters** `0x` followed by that value written out in hex — not against the **32 raw bytes** themselves. Most wallets and libraries sign the **raw** bytes (or a hash treated as raw bytes). So anything that assumes "on-chain check = same bytes we signed off-chain" will **fail** unless every signer uses this exact hex-string rule. Teams can also **think** they are securing a hash when they are really securing a different byte string.

`push-chain-node/precompiles/usigverifier/query.go`

```
msgStr := "0x" + hex.EncodeToString(msg) // Convert the message to
a hex string
msgBytes := []byte(msgStr) // Convert the message string to origina
l signed bytes

ok = ed25519.Verify(pubKeyBytes, msgBytes, signature)
```

The public interface still names the second argument like a normal 32-byte message, which reads as "verify this digest" to anyone building contracts or off-chain signers.

Contracts can believe they only accept a signature over a given 32-byte hash, while the built-in check actually requires a signature over the **UTF-8 bytes of** `0x` plus the hex form of that hash. Ordinary signing stacks will not produce a passing signature without a custom step, so honest users see **rejections** and flows that depend on this verifier can **stall**. Any design that joins Push with Solana or another Ed25519 chain must spell out the same signing bytes everywhere (relayer, wallet, and contract); if they do not, relayers and on-chain checks can disagree in hard-to-spot ways. In short, the harm is broken or stuck verification, wrong trust in "we verified the hash," and cross-chain setups that drift unless everyone follows the non-obvious hex-string rule.

Status: Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 2/5

Severity: Low

Recommendations

Remediation:

1. **Preferred:** Add another entry point (for example `verifyEd25519Message`) that checks signatures over the **raw** message bytes (or over an explicit `bytes` blob with a clear naming convention), and document whether the old hex-string behavior stays for backward compatibility.
2. **Documentation:** If the hex-string behavior is intentional, say so in the contract interface docs and integration guides so every signer uses the same bytes.
3. **Tests:** Add fixed test vectors that show "sign raw 32-byte digest" vs "sign `0x` + hex as text" and what each API expects

Resolution: This issue has been addressed and resolved in commit 5659657.

F-2026-17736 - Derived call can commit state even when commit=false - Low

Description:

`DerivedEVMCallWithData` accepts a `commit` flag but calls `ApplyMessageWithConfig` with a hardcoded `true` and then commits cache context on success. This means the function can persist state even when callers pass `commit=false`.

```
tmpCtx, commitState := ctx.CacheContext()

// pass true to commit the StateDB
res, err := k.ApplyMessageWithConfig(tmpCtx, msg, nil, true, cfg, t
xConfig)
if err != nil {
return nil, err
}

if !res.Failed() {
commitState()
}
```

Any integration expecting non-mutating behavior from `commit=false` on derived execution can accidentally mutate chain state.

Assets:

- https://github.com/pushchain/push-chain-evm/

Status: Fixed

Classification

Impact Rate: 4/5

Likelihood Rate: 1/5

Severity: Low

Recommendations

Remediation:

- Pass `commit` into `ApplyMessageWithConfig`.
- Gate `commitState()` behind `if commit && !res.Failed()`.
- Add regression tests for both `commit=true` and `commit=false`.

Resolution: This issue has been addressed in commit f7b229f5, which wires the `commit` flag through `DerivedEVMCallWithData` and adds regression tests to ensure `commit=false` does not persist state.

F-2026-17738 - Reverted derived execution still emits events and mutates block bloom - Low

Description:

When `commit=true`, the code emits `ethereum_tx`/`tx_log` events and updates block bloom inside the same branch, even if `res.Failed()` is true. Since cache commit is skipped on failed execution, emitted metadata can represent uncommitted state.

```
// Emit events and log for the transaction if it is committed
if commit {
// ...
if res.Failed() {
attrs = append(attrs, sdk.NewAttribute(types.AttributeKeyEthereumTx
Failed, res.VmError))
}
// ...
ctx.EventManager().EmitEvents(sdk.Events{
sdk.NewEvent(types.EventTypeEthereumTx, attrs...),
sdk.NewEvent(types.EventTypeTxLog, txLogAttrs...),
})
// ...
k.SetBlockBloomTransient(ctx, bloomReceipt.Big())
k.SetLogSizeTransient(ctx, (k.GetLogSizeTransient(ctx))+uint64(len(
logs)))
}
```

Clients can observe logs/receipts/bloom effects for executions that did not commit state, leading to inconsistency across explorers, traces, and downstream indexers.

Assets:

- https://github.com/pushchain/push-chain-evm/

Status: Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Severity: Low

Recommendations

Remediation:

- Emit derived events and mutate bloom only under `commit && !res.Failed()`.
- Add tests ensuring failed derived execution does not publish log/bloom side effects.

Resolution: This issue has been addressed in commit 1a8d637, which gates block bloom and log-size updates on successful derived execution and adds regression tests ensuring reverted derived calls emit no log attributes while preserving `ethereum_tx` / `tx_log` event pairing.

F-2026-17740 - KV indexer path misses derived transactions - Low

Description:

The KV indexer persists Ethereum transaction metadata for accelerated JSON-RPC resolution. Indexing is gated on `isEthTx(tx)`, which requires an Ethereum extension option and an embedded `MsgEthereumTx`. Derived transactions—EVM executions initiated internally by Cosmos modules and recorded exclusively as Tendermint event attributes—do not satisfy this predicate and are therefore excluded from the index.

At query time, the RPC backend delegates to the indexer when one is configured. On a cache hit, the indexer response is returned without supplementary event parsing. On a miss for a derived transaction hash, no fallback path reconstructs the transaction from block events. The execution occurred on-chain and the events are present in the block result, but indexer-backed nodes cannot resolve the transaction through standard `eth_getTransactionByHash` and related APIs.

`push-chain-evm/indexer/kv_indexer.go`

```
if !isEthTx(tx) {
continue
}
```

`push-chain-evm/rpc/backend/tx_info.go`

```
if b.indexer != nil {
txRes, err := b.indexer.GetByTxHash(hash)
if err != nil {
return nil, nil, err
}
return txRes, nil, nil
}
```

Nodes operating with the KV indexer enabled exhibit incomplete or failed responses for derived transaction lookups, while nodes without the indexer may resolve the same hash via event reconstruction. This deployment-dependent behavior violates JSON-RPC response consistency expectations and impairs integrators, block explorers, and indexing infrastructure that assume uniform transaction availability across node configurations.

Assets:

- https://github.com/pushchain/push-chain-evm/

Status: Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 2/5

Severity: Low

Recommendations

Remediation:

- Extend the KV indexer ingestion pipeline to parse and persist derived transaction records from Tendermint execution events, using the same field schema applied to standard `MsgEthereumTx` entries.
- Alternatively, implement a fallback in the RPC retrieval path: when the indexer returns a miss or lacks derived-transaction metadata, delegate to the existing event-query reconstruction logic before returning a not-found response.

Resolution: This issue has been addressed in commit 4b0cc66e, which indexes derived EVM transactions in the KV indexer, rebuilds their RPC fields from events on indexer hits, and falls back to event-query reconstruction on indexer misses for backward compatibility.

F-2026-17745 - Derived transactions share a constant txIndex value - Low

Description:

During derived transaction event emission, the keeper attaches a transaction index attribute using the compile-time constant `DerivedTxIndex` (9999) for every internal EVM execution, regardless of its position within the block's EVM message sequence. The constant functions as a placeholder indicating non-standard indexing rather than a unique per-block ordinal.

When multiple derived executions occur within a single block, each emits an identical index value. RPC endpoints that resolve transactions by block number and index (`eth_getTransactionByBlockNumberAndIndex`), as well as any ordering or deduplication logic keyed on transaction index, cannot distinguish among co-located derived transactions.

`push-chain-evm/x/vm/types/events.go`

```
const (
DerivedTxIndex = 9999
DerivedTxType = 99
)
```

`push-chain-evm/x/vm/keeper/call_evm.go`

```
// add event for index of valid ethereum tx; NOTE: default txindex
for derivedTx
sdk.NewAttribute(types.AttributeKeyTxIndex, strconv.FormatUint(type
s.DerivedTxIndex, 10)),
```

Index collision introduces ambiguity in transaction retrieval, block ordering, and explorer presentation for derived executions. In blocks containing multiple derived transactions or a mixture of standard and derived messages, index-based queries may return incorrect results or fail to resolve the intended transaction. Severity increases with block throughput and the frequency of module-initiated EVM calls.

Assets:

- https://github.com/pushchain/push-chain-evm/

Status: Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Severity: Low

Recommendations

Remediation:

- Assign a unique, monotonically increasing Ethereum transaction index to each derived execution within a block, consistent with the indexing scheme applied to standard `MsgEthereumTx` messages.
- Align event emission in `call_evm.go` with the `EthTxIndex` semantics consumed by the RPC layer and KV indexer.

Resolution: This issue has been addressed in commit a6e46dfb, which replaces the hardcoded `DerivedTxIndex` with the shared block-level transient counter so each derived execution receives a unique, monotonically increasing Ethereum transaction index consistent with standard `MsgEthereumTx` indexing.

F-2026-17752 - Derived RPC transaction uses GasUsed as gas field - Low

Description:

The JSON-RPC transaction object schema defines the `gas` field as the transaction gas limit—the maximum gas units the sender authorized for execution. Actual gas consumption is reported separately in the transaction receipt via the `gasUsed` field.

In the derived transaction reconstruction path within `rpc/types/utils.go`, the `Gas` field of the returned `RPCTransaction` is populated from `txAdditional.GasUsed` rather than the corresponding gas limit attribute. This inverts the semantic contract of the field: clients receive execution consumption data where limit data is expected. The assignment is deterministic for all derived transaction RPC responses constructed through this code path.

`push-chain-evm/rpc/types/utils.go`

```
result := &RPCTransaction{
Type: hexutil.Uint64(txAdditional.Type),
From: common.HexToAddress(msg.From),
Gas: hexutil.Uint64(txAdditional.GasUsed),
GasPrice: (*hexutil.Big)(baseFee),
Hash: common.HexToHash(msg.Hash),
```

Downstream consumers—including wallet software, block explorers, indexing services, and gas analysis tooling—that interpret the `gas` field as a limit will derive incorrect metadata for derived transactions. Gas utilization ratios, headroom calculations, and fee estimation logic applied uniformly across transaction types will produce erroneous results for this transaction class.

Assets:

- https://github.com/pushchain/push-chain-evm/

Status: Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 2/5

Severity: Low

Recommendations

Remediation:

- Populate the `Gas` field from `txAdditional.GasLimit` or the equivalent gas-limit event attribute recorded during execution.
- Restrict gas consumption reporting to receipt-level fields (`gasUsed`) in accordance with the Ethereum JSON-RPC specification.

Resolution: This issue has been addressed in commit 7062e604, which sets the derived transaction RPC `gas` field from the recorded gas limit (`txGasLimit`) instead of `gasUsed`, with a fallback to `gasUsed` only for legacy derived txs that did not emit a gas limit attribute.

F-2026-17754 - TraceTransaction mixes Cosmos and Ethereum index domains - Low

Description:

The `TraceTransaction` implementation reconstructs the predecessor transaction set required for accurate EVM state replay. The outer predecessor loop bounds its iteration by `transaction.TxIndex`, which represents the ordinal position of the enclosing Cosmos transaction within the Tendermint block. Within each iteration, the loop invokes `GetTxByTxIndex`, which resolves transactions by Ethereum-level transaction index—a distinct coordinate space that enumerates EVM messages across the block's execution ordering.

These index domains diverge whenever a Cosmos transaction contains zero, one, or multiple EVM messages, or when derived transactions exist only as event records rather than embedded messages. Applying a Cosmos transaction ordinal as the upper bound for Ethereum-indexed lookups produces an incorrect predecessor set, potentially omitting relevant state-modifying executions or including unrelated transactions.

`**push-chain-evm/rpc/backend/tracing.go**`

```
var predecessors []*evmtypes.MsgEthereumTx
for i := 0; i < int(transaction.TxIndex); i++ {
predecessorTx, txAdditional, err := b.GetTxByTxIndex(blk.Block.Heig
ht, uint(i))
```

Trace output may reflect an execution context that diverges from the actual on-chain state at the point of target transaction execution. Debugging workflows, incident forensics, and automated trace analysis that depend on `debug_traceTransaction` may produce misleading call stacks, incorrect storage values, and erroneous gas accounting. The defect manifests in blocks with heterogeneous transaction compositions containing both standard and derived EVM executions.

Assets:

- https://github.com/pushchain/push-chain-evm/

Status: Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 2/5

Severity: Low

Recommendations

Remediation:

- Construct the predecessor set from a single canonical ordering source—the parsed per-block Ethereum message list derived from both embedded messages and event-reconstructed transactions.
- Eliminate cross-domain index translation between Cosmos transaction ordinals and Ethereum transaction index APIs within the trace reconstruction path.

Resolution: This issue has been addressed in commit 438559e6, which rebuilds `TraceTransaction` predecessor assembly using `EthTxIndex` (Ethereum execution order) instead of Cosmos `TxIndex`, decodes predecessor Cosmos txs via the resolved slot index, and adds derived-tx-specific handling so mixed standard/derived blocks trace without incorrect predecessors or panics.

F-2026-17758 - Missing upstream security patch train after fork point - Low

Description:

The Push Chain EVM fork diverged from an earlier snapshot of the upstream `cosmos/evm` repository and has not incorporated the subsequent security and correctness patch train released after the fork point. The resulting version gap encompasses a substantial set of upstream commits addressing audit findings, consensus safety, IBC callback integrity, fee market hardening, StateDB correctness, ante handler validation, precompile compatibility, and JSON-RPC/tracing defects.

The commit register below enumerates upstream fixes not yet integrated, organized by functional domain with direct references to the source commits.

Deferred integration

- `43b70ee6` — Non-deterministic EVM pre-blocker correction.

Sherlock audit remediation

- `322cff37` — Post-audit remediation batch 1.
- `369defb9` — Post-audit remediation batch 3.
- `3e8dc586` — Post-audit remediation batch 2.
- `4f565128` — Final audit patch from evm-internal review.
- `84178554` — Apply missing audit remediation changes.
- `d360903c` — Post-audit remediation batch 5.
- `d7d62cd5` — Post-audit remediation batch 4.

Consensus and transaction policy

- `0517dcf0` — Revert "(chore): move feemarket update BaseFee to EndBlock (#190)".
- `18ad77e2` — Enforce EIP-2681 nonce upper bound in ante handler.
- `1d8b9e1e` — remove allow-unprotected-txs (non-EIP-155) from vm params.
- `76819bae` — enforce one EVM tx per Cosmos tx.
- `cffad658` — non-determinism fix.

IBC and ERC20 callbacks

- `0310aa96` — add missing ValidateBasic check in ICS02 precompile.
- `0e511d32` — correct ICS20 balance accounting.
- `1fa87d91` — fix zero gas config in IBC transfer keeper.
- `20c00074` — block nested ICS20 forwarding in source callbacks.
- `727f3b9d` — fix ICS02 and ICS20 gas calculation.
- `776462c6` — fix error acknowledgement in IBC v2 middleware.

- `810e9ef0` — fix IBC token decimals query revert condition.
- `83a3a6c7` — gas accounting fix in IBC callback.
- `91519e45` — fix IBC middleware sender verification.
- `b0e16435` — reorder ERC20 IBC callback logic.
- `ccaee527` — prevent arbitrary source callback execution.

Fee market and coin metadata

- `1fd8ef37` — fix missing `evmCoinInfo` in historical state.
- `2a9e6879` — avoid nil-pointer panic when BaseFee is zero.
- `491aa78f` — respect `NoBaseFee` in dynamic fee checker.
- `6de21f76` — avoid nil-pointer when RPC runs before evmCoinInfo init.
- `7998121a` — fix denom exponent checks.
- `a48de71c` — validate decimals before conversion in `GetBaseFee`.
- `e7835b93` — align units in base fee calculation.
- `f4637461` — add `InitEvmCoinInfo` upgrade guard.

StateDB and execution path

- `008c1711` — snapshot locked balance on StateDB account.
- `264aa70f` — harden StateDB balance and event amount handling.
- `c0f47851` — fix invalid message index errors in block RPCs with StateDB commit-error txs.

Ante and account handling

- `16ec2d0e` — fix non-EIP-155 signer panic.
- `8156e38e` — fix invalid pubkey address type handling.
- `89bb3819` — fix EOA vs contract-account identification.
- `9ddd976e` — include missing ante unit test suite.
- `befde4fa` — delete EVM instance in ante handler.

Precompiles

- `09e1895d` — fix gov precompile interface typo.
- `19992de1` — fix SDK-inconsistent distribution precompile method name.
- `31a6edf9` — fix evidence precompile time type.
- `4b72cd52` — fix SDK-incompatible slashing precompile types.
- `679305dc` — apply journal-based revert approach.
- `76d8d108` — return data on precompile revert.
- `82023069` — support dynamic precompiles with `eth_call` overrides.
- `8c0d0e75` — fix precompile structure for local node script.
- `a3a2a068` — fix ERC20 precompile event emission.
- `c5ca2676` — align precompiles map with static availability checks.
- `e2930f63` — avoid nil-pointer in gov precompile response conversion.
- `ebcaefa6` — use address codecs in precompiles.

Assets:

- https://github.com/pushchain/push-chain-evm/

Status: Mitigated

Classification

Impact Rate: 2/5

Likelihood Rate: 2/5

Severity: Low

Recommendations

Remediation:

- **Preferred approach:** Merge a recent stable release branch from `cosmos/evm`, then re-apply Push-specific custom modifications atop the updated baseline.
- **Interim approach:** Cherry-pick high-priority commit groups in order of risk: Sherlock audit remediations, non-determinism corrections, IBC callback safety patches, and StateDB/RPC hardening fixes.
- Maintain a patch integration ledger tracking each upstream commit with fields for integration status, test evidence, and ownership assignment.

Resolution: This issue has been substantially addressed in commit 96231e7, which merges upstream `cosmos/evm` v0.5.1 and re-applies Push-specific customizations, covering the highest-priority upstream security fixes (including Sherlock audit and consensus/tx-policy patches); however, some later upstream fixes that landed after v0.5.1 (mainly IBC, StateDB/RPC, and fee-market) remain unintegrated and should be tracked in a follow-up upgrade to v0.7.0 or current upstream/main.

F-2026-17775 - Inconsistent log reporting for failed derived transactions across JSON-RPC endpoints - Low

Description:

Push Chain represents module-initiated EVM executions as derived transactions recorded through ABCI events rather than embedded `MsgEthereumTx` messages. When a derived execution reverts with state persistence enabled, the keeper may still emit `ethereum_tx` and `tx_log` events even though the underlying EVM state transition is not committed. Under certain Universal Executor inbound flows, the enclosing Cosmos transaction nevertheless completes successfully, leaving those events available in block results.

The JSON-RPC layer handles failed derived transactions inconsistently across query methods. `GetTransactionReceipt` parses logs from execution events without regard to failure status. `GetTransactionLogs` returns an empty result whenever the resolved transaction is marked failed, without consulting the event stream. Consequently, `eth_getTransactionReceipt` and log-specific queries for the same transaction hash can return materially different results.

The defect manifests on inbound processing paths that treat EVM execution failure as a recoverable outcome within an otherwise successful Cosmos transaction. In these flows, execution errors from derived calls are recorded in application state but are not propagated as Cosmos message failures. Event attributes produced during the reverted EVM invocation—including transaction logs—therefore remain indexed in the block result.

On execution paths that return an error to the Cosmos message handler, the enclosing transaction fails and associated events are not retained. The inconsistency described herein primarily affects inbound gas and funds workflows that complete at the Cosmos layer despite EVM-level revert.

Receipt construction — logs parsed irrespective of failure status

`push-chain-evm/rpc/backend/tx_info.go`

```
msgIndex := int(res.MsgIndex) // #nosec G115 -- checked for int ove
rflow already
logs, err := TxLogsFromEvents(blockRes.TxsResults[res.TxIndex].Even
ts, msgIndex)
if err != nil {
b.logger.Debug("failed to parse logs", "hash", hexTx, "error", err.
Error())
}
```

Dedicated log query — empty response on failure

`push-chain-evm/rpc/backend/tx_info.go`

```
if res.Failed {
// failed, return empty logs
```

```
return nil, nil
}
```

Inbound handler — execution error not propagated to Cosmos message result

`push-chain-node/x/uexecutor/keeper/execute_inbound_gas.go`

```
// Never return execErr, only nil
return nil
```

Block explorers, indexing infrastructure, and client applications that source log data from `eth_getTransactionReceipt` may ingest revert-associated logs that dedicated log queries omit for the identical transaction hash. This violates the expectation—observed on Ethereum mainnet—that failed executions do not yield divergent log representations across standard JSON-RPC interfaces.

Where reverted derived executions emit log events without a corresponding state commit, receipt-level log data may not reflect durable on-chain execution outcomes. Integrators operating dual ingestion paths or cross-validating receipt and log endpoints are exposed to silent data inconsistency without error indication.

Assets:

- https://github.com/pushchain/push-chain-evm/

Status: Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Severity: Low

Recommendations

Remediation: At the execution layer, restrict derived transaction event emission—including `tx_log` attributes and bloom updates—to successful EVM invocations only. At the RPC layer, apply uniform failure semantics across `GetTransactionReceipt` and `GetTransactionLogs`, such that failed transactions return empty log sets through both interfaces. Introduce integration coverage for inbound revert scenarios in which the Cosmos transaction succeeds, verifying consistent log responses across all supported query methods.

Resolution: This issue has been addressed in commit 4e1260cb.

F-2026-16598 - Unsafe Status Modification Order in MarkBallotExpired() and MarkBallotFinalized() - Info

Description:

Ballot `Status` is modified and persisted to storage before all collection operations complete.
The '`MarkBallotExpired'` function in '`x/uvalidator/keeper/ballot.go'` (lines 114-133) removes a ballot ID from '`ActiveBallotIDs`' without first verifying the ID is currently present in the set. Similarly, '`MarkBallotFinalized'` (lines 137-160) has the same issue. If either function is invoked multiple times for the same ballot (e.g., due to network retries or transaction replays), the second call will fail because it attempts to remove an already-removed key.

The effects is that the Ballot is marked `EXPIRED` in storage, but not removed from `ActiveBallotIDs` and not added to `ExpiredBallotIDs`. The ballot becomes "orphaned" in an inconsistent state.

Status: Fixed

Classification

Severity: Info

Recommendations

Remediation: Only set the Ballot after all checks are done:

```
func (k Keeper) MarkBallotExpired(ctx context.Context, id string) e
rror {
// ... get ballot ...

// Only modify and persist atomically with all other operations
if err := k.ActiveBallotIDs.Remove(ctx, id); err != nil {
return err
}
if err := k.ExpiredBallotIDs.Set(ctx, id); err != nil {
return err
}

ballot.Status = types.BallotStatus_BALLOT_STATUS_EXPIRED
return k.Ballots.Set(ctx, id, ballot)
}
```

Resolution: This issue has been addressed and resolved in commit 4d9c8ee.

F-2026-16614 - Centralized registry and Universal Validator roster - Info

Description:

Chain and token configuration under `x/uregistry` is governed by administrative parameters and governance. The Universal Validator roster is maintained under `x/uvalidator`. Inbound voting and user execution paths consult registry flags such as whether inbound transfers are enabled for a given chain. Supported routes may therefore be disabled or reconfigured without individual user consent, which affects new activity and may exclude specific assets or chains according to policy.

Users may find that new bridging or execution stops abruptly when governance or administrators change registry settings. Operators must align incident response and communications with governance outcomes and on-chain configuration snapshots. From a compliance and product perspective, centralized route control can be desirable or sensitive depending on the regulatory environment. It should be disclosed plainly rather than implied.

Status: Accepted

Classification

Severity: Info

Recommendations

Remediation:

Document for end users and integrators that route availability is governance- and admin-controlled. Use time-locks, multisig thresholds, and transparent changelogs for registry updates where appropriate. Describe exit or recovery paths when a chain is disabled, and add monitoring that surfaces inbound-enablement changes for operators.

F-2026-16616 - Chain Metadata Oracle Using First Vote Instead of Median Aggregation - Info

Description:

When no `ChainMeta` record exists for `observedChainId`, the first successful `VoteChainMeta` path invokes `CallUniversalCoreSetChainMeta` immediately with that validator's price and block height, then persists a single-voter state. Median aggregation, staleness filtering, and stricter height rules apply only after initialization. The first writer therefore temporarily defines on-chain oracle values without multi-validator aggregation.

Contracts or applications that consume chain metadata may observe briefly skewed gas prices or heights during oracle bootstrap. DeFi components that price or time operations from this feed inherit that cold-start sensitivity. The first validator to succeed on a new `observedChainId` exerts disproportionate influence unless governance seeds values or multiple votes are required first.

Status: Fixed

Classification

Severity: Info

Recommendations

Remediation: Implement multiple agreeing votes or a governance-supplied seed before the first contract write, or defer the EVM update until a median can be computed from an initial batch. Document cold-start semantics for integrators. Add property or integration tests for the transition from empty metadata to multi-voter median behavior.

Resolution: This issue has been addressed and resolved in commit 8848c0a.

F-2026-16619 - ExecutePayload and binding of UniversalAccountId.owner to the signer - Info

Description:

The `ExecutePayload` implementation does not require that `UniversalAccountId.Owner` equal the EVM address derived from `MsgExecutePayload.Signer`. The factory resolves the target UEA from the entire `UniversalAccountId` embedded in the transaction, while `CallUEAExecutePayload` sends the call with `evmFrom` computed solely from `Signer`. Neither the keeper nor the gRPC handler introduces an equality predicate linking those two identities.

Accordingly, who signs the Cosmos transaction and which owner the message names may diverge without rejection at this layer. Whether that divergence is benign (e.g., sponsored execution with off-chain proof) or hazardous is determined only by the UEA's `executeUniversalTx` implementation and the semantics of `verificationData`.

Status: Accepted

Classification

Severity: Info

Recommendations

Remediation:

1. For paths where `Owner` must equal `Signer`, enforce equality in `ValidateBasic`, the handler, or the keeper (normalize addresses before comparison).
2. If relay remains supported, distinguish message variants or flags so unchecked divergence is explicit at the API surface.
3. Exercise UEA `executeUniversalTx` under `Owner ≠ evmFrom` with crafted `verificationData` in unit and integration tests.
4. Publish the authoritative rule: module-enforced binding vs contract-only binding.

F-2026-16648 - Default registry administrator in DefaultParams - Info

Description:

`x/uregistry` `DefaultParams` returns a fixed bech32 administrator address alongside a `TODO` marker. `Validate` rejects only an empty administrator string. If genesis or migrations omit an override, that default address controls registry-gated messages until changed through authorized flows, which is undesirable for production networks.

`x/uregistry/types/params.go` — `DefaultParams`,

```
// DefaultParams returns default module parameters.
func DefaultParams() Params {
// TODO:
return Params{
Admin: "push1negskcfqu09j5zvpk7nhvacnwyy2mafffy7r6a",
}
}
```

`x/uregistry/types/params.go` — `Validate`,

```
func (p Params) Validate() error {
if strings.TrimSpace(p.Admin) == "" {
return fmt.Errorf("admin address cannot be empty")
}
return nil
}
```

Improperly configured genesis files for testnets or mainnet deployments can unintentionally grant elevated privileges to unauthorized keys. Furthermore, reliance on hardcoded default addresses introduces ambiguity between development and production environments, increasing the risk of misconfiguration during security reviews.

Status: Fixed

Classification

Severity: Info

Recommendations

Remediation: Always set the registry administrator explicitly in production genesis. Prefer defaults that cannot be mistaken for production keys, or fail genesis validation unless an administrator is supplied. Add continuous integration checks on generated genesis artifacts for expected multisig administrators.

Resolution: This issue has been addressed and resolved in commit 2003414

F-2026-16655 - Fund migration votes pass txHash without normalization - Info

Description:

`MsgVoteOutbound` strips an optional `0x` prefix from transaction identifiers before voting. `MsgVoteFundMigration` forwards `txHash` unchanged into the ballot key helper. Validators that report the same logical transaction with and without the `0x` prefix therefore hash different ballot identifiers, splitting quorum in the same structural class of issue as serialization-sensitive keys elsewhere.

Outbound normalization occurs in the executor message server.

File: `push-chain-node/x/uexecutor/keeper/msg_server.go` .

```
// Normalize IDs: strip 0x prefix
msg.TxId = strings.TrimPrefix(msg.TxId, "0x")
msg.UtxId = strings.TrimPrefix(msg.UtxId, "0x")
```

Fund migration passes the raw message field through to the keeper without altering `TxHash`.

File: `push-chain-node/x/utss/keeper/msg_server.go` .

```
err = ms.k.VoteFundMigration(ctx, signerValAddr, msg.MigrationId, m
sg.TxHash, msg.Success)
if err != nil {
return nil, err
}
```

The ballot key concatenates the migration identifier, raw hash string, and success flag before hashing.

File: `push-chain-node/x/utss/types/keys.go` .

```
func GetFundMigrationBallotKey(migrationId uint64, txHash string, s
uccess bool) string {
canonical := fmt.Sprintf("fm:%d:%s:%t", migrationId, txHash, succes
s)
h := sha256.Sum256([]byte(canonical))
```

Fund migration finalization can stall despite off-chain agreement because on-chain ballots never merge. Operators waste time reconciling formatting rather than chain facts. Users experience delayed or stuck migration completion.

Status: Accepted

Classification

Severity: Info

Recommendations

Remediation: Canonicalize `txHash` in the fund migration message server before voting—strip optional `0x`, lowercase hexadecimal for EVM hashes, and reject malformed lengths—reusing shared helpers with outbound handling. Add parameterized tests that enumerate common alternate representations of the same logical hash. Document the canonical format for Universal Validator operators.

F-2026-16867 - Logging and standard output disclose sensitive material - Info

Description:

The Universal Client exposes sensitive operational data through two channels that are commonly aggregated, retained, and forwarded to centralized monitoring. First, the `puniversald start` command serializes the loaded `Config` structure to JSON and writes it to standard output before initialization completes. That output includes `keyring_password`, `tss_password`, and `tss_p2p_private_key_hex`, which therefore appear in process output captured by init systems, container runtimes, and log shipping pipelines on every restart. Whatever values are present in the loaded configuration file—whether initial defaults or operator-supplied replacements are serialized in full on each launch. Second, upon completion of TSS signing and key-handling sessions, the session manager records hex-encoded signatures, public keys, and (for key sessions) a hash of the keyshare, placing per-event cryptographic material into the same structured log streams. Either channel increases the consequence of insufficient access controls on log storage, export policies, or diagnostic bundles.

The start path loads configuration from disk and prints the full marshalled document without redaction.

`/cmd/puniversald/commands.go`

```
loadedCfg, err := uvconfig.Load(home)
if err != nil {
return fmt.Errorf("failed to load config: %w", err)
}

configJSON, err := json.MarshalIndent(loadedCfg, "", " ")
if err != nil {
return fmt.Errorf("failed to marshal config: %w", err)
}
fmt.Printf("\n=== Loaded Configuration ===\n%s\n==================
========\n\n", string(configJSON))
```

The `Config` type carries credentials and key material with ordinary JSON field names, so they serialize verbatim.

`/universalClient/config/types.go`

```
// Keyring
KeyringBackend KeyringBackend `json:"keyring_backend"`
KeyringPassword string `json:"keyring_password"`
// ...
// TSS
TSSP2PPrivateKeyHex string `json:"tss_p2p_private_key_hex"`
TSSP2PListen string `json:"tss_p2p_listen"`
TSSPassword string `json:"tss_password"`
```

Disclosure of the keyring password and TSS password expands the attack surface for offline attempts against the keyring and against keyshare encryption, subject to the configured backend and filesystem protections.

Disclosure of the TSS libp2p private key in hexadecimal form permits impersonation of the validator's peer identity on the TSS network for any party that obtains log or stdout access without direct host compromise.

After outbound signing completes, the session manager logs the signature and public.

`/universalClient/tss/sessionmanager/sessionmanager.go`

```
func (sm *SessionManager) handleSignFinished(ctx context.Context, e
ventID string, result *dkls.Result, signingReq *common.UnsignedSign
ingReq) error {
sm.logger.Info().
Str("event_id", eventID).
Str("signature", hex.EncodeToString(result.Signature)).
Str("public_key", hex.EncodeToString(result.PublicKey)).
Msg("signature generated from sign session")
```

Key protocol completion records the public key and a SHA-256 hash of the keyshare (the hash does not reveal the keyshare itself but still constitutes operational metadata at elevated sensitivity).

`/universalClient/tss/sessionmanager/sessionmanager.go`

```
keyshareHash := sha256.Sum256(result.Keyshare)
sm.logger.Info().
Str("event_id", eventID).
Str("protocol", protocolType).
Str("storage_id", storageID).
Str("public_key", hex.EncodeToString(result.PublicKey)).
Str("keyshare_hash", hex.EncodeToString(keyshareHash[:])).
Msg("saved keyshare")
```

A produced signature does not, by itself, authorize arbitrary new transactions without the corresponding signing key material however, persisting full signature values alongside identifiers at default severity elevates the confidentiality requirements on log repositories and may conflict with policies that restrict distribution of transaction-correlated cryptographic artifacts. Exposure facilitates correlation and forensic analysis by any principal with log read access beyond the minimum required for operations.

Status: Fixed

Classification

Severity: Info

Recommendations

Remediation: Production builds should omit unconditional printing of the full configuration, or should emit a redacted summary that masks password fields and truncates or omits hex-encoded secrets. Credential delivery via environment variables or a secrets manager, with explicit policy against

echoing those values, aligns with common operational controls. Any diagnostic mode that reproduces full configuration should be disabled by default (for example behind an explicit `--print-config` flag), documented as unsuitable for production, and restricted to controlled environments. Logging should prefer identifiers such as `event_id` and, once available, the transaction hash, while relegating full signature octets to Debug level or omitting them from automated pipelines. Where retention of signature material is required for incident response, organizations may scope retention to restricted indices, role-based access, or complementary controls on fields that carry cryptographic content.

Resolution: This issue has been addressed and resolved in commit a00e7d6

F-2026-16877 - Chains.Start returns successfully when the Push native chain client is not attached - Info

Description: During startup, `ensurePushChain` is invoked on a best-effort basis. If it fails, the implementation logs a warning and `Start` still returns a nil error. A background goroutine continues to call `fetchAndUpdate`, which retries attachment of the Push native client; until that succeeds, the process may run with external chain clients active while the Push native chain client is absent. Monitoring that infers full observation capability from process or generic health status alone may not detect that Push-originated ingress and egress events are not yet processed unless dedicated readiness checks exist.

`/universalClient/chains/chains.go`

```
// Always create push chain client first
if err := c.ensurePushChain(ctx); err != nil {
c.logger.Warn().Err(err).Msg("failed to create push chain client; c
ontinuing")
}

go c.run(ctx)
return nil
```

Votes or processing for work scoped to the Push native client may be omitted or delayed until a later synchronization cycle succeeds.

Status: Fixed

Classification

Severity: Info

Recommendations

Remediation: Provide a strict startup mode that exits with a non-zero status when the Push chain client is mandatory expose an explicit readiness signal so orchestration can wait until native observation is active.

Resolution: This issue has been addressed and resolved in commit 634e234

F-2026-16994 - Nil Network on MsgUpdateUniversalValidator panics in validation and in the handler - Info

Description:

`MsgUpdateUniversalValidator` carries network metadata as an optional protobuf pointer. `ValidateBasic` forwards directly to `msg.Network.ValidateBasic()` without checking for nil, and the message server dereferences `msg.Network` when calling the keeper. A nil network therefore panics in the usual ante path or, if validation were bypassed, on the first keeper call. The add-validator message in the same module already returns `ErrInvalidRequest` when `Network` is nil, so the update path is inconsistent and likely an oversight rather than intentional.

File: `push-chain-node/x/uvalidator/types/msg_update_universal_validator.go`

```
func (msg *MsgUpdateUniversalValidator) ValidateBasic() error {
if _, err := sdk.AccAddressFromBech32(msg.Signer); err != nil {
return errors.Wrap(err, "invalid signer address")
}

return msg.Network.ValidateBasic()
}
```

File: `push-chain-node/x/uvalidator/keeper/msg_server.go`

```
func (ms msgServer) UpdateUniversalValidator(ctx context.Context, m
sg *types.MsgUpdateUniversalValidator) (*types.MsgUpdateUniversalVa
lidatorResponse, error) {
// ...
validator, err := ms.k.StakingKeeper.GetValidator(ctx, valAddr)
if err != nil {
return nil, errors.Wrap(err, "signer is not a validator operator")
}

err = ms.k.UpdateUniversalValidator(ctx, validator.OperatorAddress,
*msg.Network)
```

File: `push-chain-node/x/uvalidator/types/msg_add_universal_validator.go`

```
func (msg *MsgAddUniversalValidator) ValidateBasic() error {
// ...
if msg.Network == nil {
return errors.Wrap(sdkerrors.ErrInvalidRequest, "network info is re
quired")
}

return msg.Network.ValidateBasic()
}
```

Clients that omit `network` receive a generic panic-recovery failure instead of a clear invalid-request error. Internal or test callers that skip `ValidateBasic` hit the dereference in the handler. The mismatch with the add message increases support burden and hides the actual validation rule.

Status: Fixed

Classification

Severity: Info

Recommendations

Remediation:

- Add the same nil guard and error message to `MsgUpdateUniversalValidator.ValidateBasic` as on the add message.
- Guard `msg.Network` in `UpdateUniversalValidator` before logging or dereferencing, and return `ErrInvalidRequest` if absent.
- Optionally change the keeper API to accept `*types.NetworkInfo` so nullability is explicit in the type system.

Resolution: This issue has been addressed and resolved in commit a4ae809

F-2026-16996 - Unwired v2 migration overwrites every universal validator network identity with placeholders - Info

Description:

A version-two migration function rewrites the universal validator map from a legacy key set into the current `UniversalValidator` structure. For every migrated row it assigns the same hardcoded libp2p peer identifier and a single loopback multiaddr. The routine is not registered on the module configurator, and consensus version remains one, so it does not run in production builds today. The hazard is forward-looking: wiring this migration without replacing the placeholders would collapse every validator's advertised network identity to localhost and a shared peer id, which would break TSS and libp2p routing across the set. The file also invites copy-paste into a future upgrade handler.

File: `push-chain-node/x/uvalidator/migrations/v2/migrate.go`

```
func MigrateUniversalValidatorSet(ctx sdk.Context, k *keeper.Keeper
, cdc codec.BinaryCodec) error {
// ...
for ; iter.Valid(); iter.Next() {
valAddr, err := iter.Key()
// ...

newVal := types.UniversalValidator{
IdentifyInfo: &types.IdentityInfo{
CoreValidatorAddress: valAddr.String(),
},
NetworkInfo: &types.NetworkInfo{
PeerId: "12D3KooWFNC8BxiPoHyTJtiN1u1ctw3nSexuJHUBv4mMMmqEtQgg",
MultiAddrs: []string{"/ip4/127.0.0.1/tcp/39001/p2p/12D3KooWFNC8BxiP
oHyTJtiN1u1ctw3nSexuJHUBv4mMMmqEtQgg"},
},
LifecycleInfo: &types.LifecycleInfo{
CurrentStatus: types.UVStatus_UV_STATUS_PENDING_JOIN,
History: []*types.LifecycleEvent{
{
Status: types.UVStatus_UV_STATUS_PENDING_JOIN,
BlockHeight: ctx.BlockHeight(),
},
},
},
}

if err := k.UniversalValidatorSet.Set(ctx, valAddr, newVal); err !=
nil {
return err
}
}
```

File: `push-chain-node/x/uvalidator/module.go`

```
const (
ConsensusVersion = 1
)
```

```
func (a AppModule) RegisterServices(cfg module.Configurator) {
types.RegisterMsgServer(cfg.MsgServer(), keeper.NewMsgServerImpl(a.
keeper))
types.RegisterQueryServer(cfg.QueryServer(), keeper.NewQuerier(a.ke
```

```
eper))
}
```

No `RegisterMigration` call registers `MigrateUniversalValidatorSet`.

If the migration were enabled as written, every validator would share one peer id and one loopback address, so handshakes and routing would fail until operators issued updates for real network parameters. Recovery would be operationally heavy and concurrent with broken TSS connectivity. While the code is dead, retaining it without clear warnings risks accidental activation during a consensus-version bump.

Status: Fixed

Classification

Severity: Info

Recommendations

Remediation:

- Replace placeholder `PeerId` and `MultiAddrs` with empty values or migration-supplied parameters read from genesis or governance, and block promotion to `ACTIVE` until real network info is present.
- Remove the migration file if it is obsolete, relying on version control for history.
- Add an integration or unit test that fails if a migration run would overwrite distinct network identities with identical literals.

Resolution: This issue has been addressed and resolved in commit [82fd7fe]

F-2026-17024 - Nil nested configs on add/update messages cause panics in validation and logging - Info

Description:

The add and update messages for chain and token configuration carry nested protobuf messages as pointers. `ValidateBasic` on each message calls `ValidateBasic` on the nested config without checking for a nil pointer first. The nested validators use value receivers and read fields immediately, so a nil pointer dereference panics during ante validation in typical flows. The message server handlers also log fields from the nested config before any nil check, which creates a second panic surface if validation were ever skipped or reordered. The practical outcome is failed transactions with panic-recovery messaging rather than a clear validation error, which harms integrators and obscures the root cause.

File: `push-chain-node/x/uregistry/types/msg_add_chain_config.go` ·

```
func (msg *MsgAddChainConfig) ValidateBasic() error {
if _, err := sdk.AccAddressFromBech32(msg.Signer); err != nil {
return errors.Wrap(err, "invalid signer address")
}

return msg.ChainConfig.ValidateBasic()
}
```

The same pattern appears in `msg_update_chain_config.go`, `msg_add_token_config.go`, and `msg_update_token_config.go` for `ChainConfig` and `TokenConfig` respectively.

File: `push-chain-node/x/uregistry/keeper/msg_server.go` ·

```
func (ms msgServer) AddChainConfig(ctx context.Context, msg *types.
MsgAddChainConfig) (*types.MsgAddChainConfigResponse, error) {
ms.k.Logger().Info("msg add chain config received", "signer", msg.S
igner, "chain", msg.ChainConfig.Chain)
```

Analogous logging appears in `UpdateChainConfig`, `AddTokenConfig`, and `UpdateTokenConfig` before any nil guard on `msg.ChainConfig` or `msg.TokenConfig`.

Malformed or adversarial clients that omit the nested message cause panics recovered by the SDK, so the transaction fails with a generic recovered-panic error instead of `ErrInvalidRequest` with an explicit “config required” message. Tests and simulations that construct messages without the nested field see panics where they expect typed errors. Any future change that bypasses `ValidateBasic` in the ante chain would hit the logging path first and panic there as well.

Status: Fixed

Classification

Severity: Info

Recommendations

Remediation:

- In each affected `ValidateBasic`, return a wrapped `ErrInvalidRequest` when `ChainConfig` or `TokenConfig` is nil before delegating to nested validation.
- At the start of each handler, validate presence of the nested message before logging its fields, and return a typed error if it is missing.
- Optionally use pointer receivers on nested `ValidateBasic` methods so nil receives an explicit error path instead of an implicit dereference panic.
- Add unit tests that submit nil nested configs and assert a non-panic validation error for all four message types.

Resolution: This issue has been addressed and resolved in commit [bf0c3e]

F-2026-17035 - Unpaginated query walks and full-collection scan in GetTokenConfigByPRC20 - Info

Description:

Three query endpoints — `AllChainConfigs`, `AllTokenConfigs`, and `TokenConfigsByChain` — walk their respective collections in full and return the materialized slice in a single response. None of the corresponding request types carries a `cosmos.base.query.v1beta1.PageRequest`, so clients have no way to limit response size. `TokenConfigsByChain` additionally throws away the chain-prefixed storage key (`<chain>:<address>`) it could otherwise scan via a prefix range, filtering by `value.Chain == req.Chain` in memory after reading every entry. The same pattern repeats inside `GetTokenConfigByPRC20`, which is invoked from outbound creation and revert resolution in `x/uexecutor`: it walks the entire `TokenConfigs` map per outbound, paying O(N_total_tokens) DB reads even though only the entries for one chain are eligible matches.

File: `push-chain-node/x/uregistry/keeper/query_server.go` ·

```
func (k Querier) AllChainConfigs(goCtx context.Context, req *types.
QueryAllChainConfigsRequest) (*types.QueryAllChainConfigsResponse,
error) {
ctx := sdk.UnwrapSDKContext(goCtx)
var configs []*types.ChainConfig
err := k.Keeper.ChainConfigs.Walk(ctx, nil, func(key string, value
types.ChainConfig) (stop bool, err error) {
v := value
configs = append(configs, &v)
return false, nil
})
// ...
return &types.QueryAllChainConfigsResponse{Configs: configs}, nil
}
```

`AllTokenConfigs` (lines 78–96) follows the identical unbounded pattern.

File: `push-chain-node/x/uregistry/keeper/query_server.go` ·

```
func (k Querier) TokenConfigsByChain(goCtx context.Context, req *ty
pes.QueryTokenConfigsByChainRequest) (*types.QueryTokenConfigsByCha
inResponse, error) {
ctx := sdk.UnwrapSDKContext(goCtx)
var configs []*types.TokenConfig
err := k.Keeper.TokenConfigs.Walk(ctx, nil, func(key string, value
types.TokenConfig) (stop bool, err error) {
if value.Chain == req.Chain {
v := value
configs = append(configs, &v)
}
return false, nil
})
// ...
}
```

The storage key already begins with `req.Chain + ":"`, so a prefix range would scope the walk to O(N_chain) instead of the current O(N_total).

File: `push-chain-node/x/uregistry/types/query.pb.go` ·

```
type QueryAllChainConfigsRequest struct {
}
// ...
type QueryAllTokenConfigsRequest struct {
}
// ...
type QueryTokenConfigsByChainRequest struct {
Chain string `protobuf:"bytes,1,opt,name=chain,proto3" json:"chain,
omitempty"`
}
```

No `Pagination` field on any of the three requests.

File: `push-chain-node/x/uregistry/keeper/keeper.go` .

```
func (k Keeper) GetTokenConfigByPRC20(
ctx context.Context,
chain string,
prc20Addr string,
) (types.TokenConfig, error) {
prc20Addr = strings.ToLower(strings.TrimSpace(prc20Addr))
var found *types.TokenConfig
err := k.TokenConfigs.Walk(ctx, nil, func(key string, cfg types.Tok
enConfig) (bool, error) {
if cfg.Chain != chain {
return false, nil
}
if cfg.NativeRepresentation == nil {
return false, nil
}
if strings.ToLower(cfg.NativeRepresentation.ContractAddress) == prc
20Addr {
found = &cfg
return true, nil
}
return false, nil
})
// ...
}
```

This is called per outbound transaction from `x/uexecutor/keeper/create_outbound.go` and `build_revert_outbound.go`, so the linear scan executes inside consensus, not only at query time.

`AllChainConfigs` and `AllTokenConfigs` are public read endpoints whose cost grows linearly with registry size; an automated crawler that hammers them produces O(N) DB I/O per request, with the only mitigation being whatever RPC-layer rate-limiting the operator configures. `TokenConfigsByChain` is O(N_total) instead of O(N_chain), so a query for a small chain still pays the cost of the largest chain's token list. The most consequential consumer is `GetTokenConfigByPRC20`: it runs in the outbound and revert paths inside a transaction context, so its O(N) cost is paid by every outbound on the bridge — block-time and gas-meter overhead scale linearly with the number of registered tokens across all chains. Adding pagination after launch is also a breaking proto change unless paired with a major version bump, which makes the omission a forward-compatibility trap as well.

Status: Fixed

Classification

Severity: Info

Recommendations

Remediation:

- Add `cosmos.base.query.v1beta1.PageRequest pagination` to `QueryAllChainConfigsRequest`, `QueryAllTokenConfigsRequest`, and `QueryTokenConfigsByChainRequest`, and switch the handlers to `query.CollectionPaginate(...)` (or the equivalent `cosmossdk.io/collections.Paginate` helper).
- Rewrite `TokenConfigsByChain` to use a prefix range over the `<chain>:` key prefix instead of a full-collection walk with an in-memory filter.
- Introduce a secondary index `PRC20Index: collections.Map[(chain, lowercased_prc20_contract), token_storage_key]`, populated and invalidated by `AddTokenConfig` / `UpdateTokenConfig` / `RemoveTokenConfig`. Rewrite `GetTokenConfigByPRC20` as a single index lookup followed by a single `TokenConfigs.Get`.
- Even without the reverse index, scope `GetTokenConfigByPRC20` to the chain prefix so the wrong-chain entries are skipped at the storage layer.
- Add a benchmark in the keeper test suite that seeds `K * M` tokens and asserts `GetTokenConfigByPRC20` reads remain constant under load.

Resolution: This issue has been addressed and resolved in commit [f409cb]

F-2026-17040 - TSS event status updates scan entire event store - Info

Description:

Updating or completing TSS-related events uses `TssEvents.Walk` with a full-store scan to locate rows by `process_id` and `event_type`, and `completePreviousActiveFinalizedEvent` walks **all** events to mark prior finalized keys completed. As the event history grows, these operations become linear in store size and are invoked from consensus-critical paths (for example TSS key finalization).

Single-row lookup by scanning all events — `push-chain-node/x/utss/keeper/tss_events.go`

```
func (k Keeper) updateTssEventStatusByProcessId(ctx context.Context
, processId uint64, eventType types.TssEventType, newStatus types.T
ssEventStatus) error {
var foundId uint64
var foundEvent types.TssEvent
found := false

err := k.TssEvents.Walk(ctx, nil, func(id uint64, event types.TssEv
ent) (bool, error) {
if event.ProcessId == processId && event.EventType == eventType {
foundId = id
foundEvent = event
found = true
return true, nil // stop walking
}
return false, nil // continue
})
```

Prior finalized events: collect all matches in one walk

```
err := k.TssEvents.Walk(ctx, nil, func(id uint64, event types.TssEv
ent) (bool, error) {
if event.EventType == types.TssEventType_TSS_EVENT_KEY_FINALIZED &&
event.Status == types.TssEventStatus_TSS_EVENT_ACTIVE {
toUpdate = append(toUpdate, struct {
id uint64
event types.TssEvent
}{id: id, event: event})
}
return false, nil // continue walking to find all
})
```

These helpers are invoked from `VoteTssKeyProcess` finalization (for example `updateTssEventStatusByProcessId` and `completePreviousActiveFinalizedEvent` in `msg_vote_tss_key_process.go`).

Long-lived networks accumulate `TssEvents`; each finalization performs O(n) work over that map. Under load, this increases transaction gas for voters and creates a mild self-DoS vector tied to history size rather than current validator count.

Status: Accepted

Classification

Severity: Info

Recommendations

Remediation:

1. Maintain a direct secondary index `(process_id, event_type) → event_id` for rows that must be updated cheaply, or store `last_process_initiated_event_id` on `TssKeyProcess`.
2. For “complete previous finalized” semantics, keep a single `current_active_finalized_event_id` pointer updated on each rotation instead of scanning all historical finalized events.
3. Add benchmarks or fuzz tests that measure gas with large `TssEvents` tables.

Observation Details

Disclaimers

Hacken Disclaimer

The blockchain protocol given for audit has been analyzed based on best industry practices at the time of the writing of this report, with cybersecurity vulnerabilities and issues in the protocol source code, the details of which are disclosed in this report (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

The report contains no statements or warranties on the identification of all vulnerabilities and security of the code. The report covers the code submitted and reviewed, so it may not be relevant after any modifications. Do not consider this report as a final and sufficient assessment regarding the utility and safety of the code, bug-free status, or any other protocol statements.

While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only — we recommend proceeding with several independent audits and a public bug bounty program to ensure the security of the blockchain protocol.

English is the original language of the report. The Consultant is not responsible for the correctness of the translated versions.

As part of Hacken's ongoing quality assurance process, we may conduct re-audits of select projects. These re-audits are performed independently from the original audit and are intended solely for internal quality control and improvement. Updated reports resulting from such re-audits will be shared privately with the respective clients and may be published on the Hacken website only with their explicit consent.
The sole authoritative source for finalized and most up-to-date versions of all reports remains the Audits section at https://hacken.io/audits/.

Technical Disclaimer

Blockchain protocols are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the protocol can have vulnerabilities that can lead to hacks. Thus, the Consultant cannot guarantee the explicit security of the audited blockchain protocol.

Appendix 1. Severity Definitions

Severity	Description
Critical	Vulnerabilities that can lead to a complete breakdown of the blockchain network's security, privacy, integrity, or availability fall under this category. They can disrupt the consensus mechanism, enabling a malicious entity to take control of the majority of nodes or facilitate 51% attacks. In addition, issues that could lead to widespread crashing of nodes, leading to a complete breakdown or significant halt of the network, are also considered critical along with issues that can lead to a massive theft of assets. Immediate attention and mitigation are required.
High	High severity vulnerabilities are those that do not immediately risk the complete security or integrity of the network but can cause substantial harm. These are issues that could cause the crashing of several nodes, leading to temporary disruption of the network, or could manipulate the consensus mechanism to a certain extent, but not enough to execute a 51% attack. Partial breaches of privacy, unauthorized but limited access to sensitive information, and affecting the reliable execution of smart contracts also fall under this category.
Medium	Medium severity vulnerabilities could negatively affect the blockchain protocol but are usually not capable of causing catastrophic damage. These could include vulnerabilities that allow minor breaches of user privacy, can slow down transaction processing, or can lead to relatively small financial losses. It may be possible to exploit these vulnerabilities under specific circumstances, or they may require a high level of access to exploit effectively.
Low	Low severity vulnerabilities are minor flaws in the blockchain protocol that might not have a direct impact on security but could cause minor inefficiencies in transaction processing or slight delays in block propagation. They might include vulnerabilities that allow attackers to cause nuisance-level disruptions or are only exploitable under extremely rare and specific conditions. These vulnerabilities should be corrected but do not represent an immediate threat to the system.

Appendix 2. Scope

The scope of the project includes the following components from the provided repository:

Scope Details	
Repository	https://github.com/pushchain/push-chain-node
Commit	ebaa89ddc70fdf9b7d486d89add0e298c44b8973
Repository	https://github.com/pushchain/push-chain-evm
Commit	9570d87d7b5eaff643885eec380b9275c85a4638

Appendix 3. Additional Valuables

Frameworks and Methodologies

This security assessment was conducted in alignment with recognised penetration testing standards, methodologies and guidelines, including the NIST SP 800-115 – Technical Guide to Information Security Testing and Assessment, and the Penetration Testing Execution Standard (PTES), These assets provide a structured foundation for planning, executing, and documenting technical evaluations such as vulnerability assessments, exploitation activities, and security code reviews. Hacken's internal penetration testing methodology extends these principles to Web2 and Web3 environments to ensure consistency, repeatability, and verifiable outcomes.